

Genetic Algorithms Course (ALGEN)

MSc in Electrical and Computer Engineering

Filipe Azevedo
(fta@isep.ipp.pt)

Program

Theoretical Program:

- Evolutionary Computation
- Introduction to Genetic Algorithms
- Swarm Intelligence
 - Particle Swarm Optimization
 - Ant Colony Optimization

Program

Practical Program:

- Optimization of functions using the Matlab Optimization toolbox
- Implementation of Genetic Algorithms in MATLAB
- Practical assignments

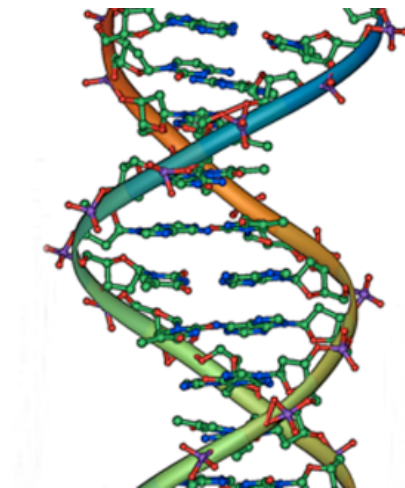
Evolutionary Computation

Genetic Algorithms Course (ALGEN)
MSc in Electrical and Computer Engineering

Filipe Azevedo
(fta@isep.ipp.pt)

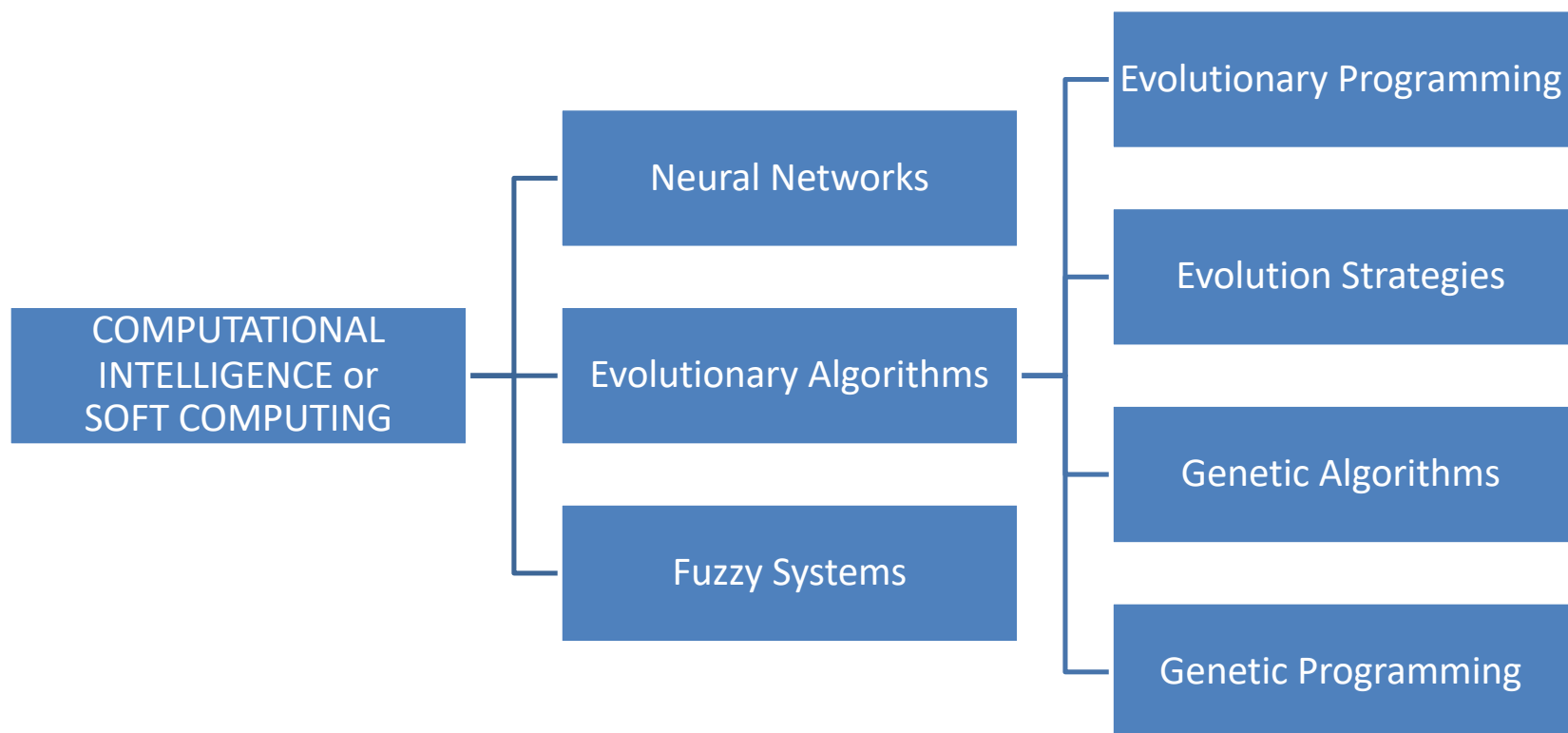
What is Evolutionary Computation?

- Solving problems with techniques inspired by biology.



- An abstraction from the theory of biological evolution that is used to create optimization procedures or methodologies, usually implemented on computers, that are used to solve problems.

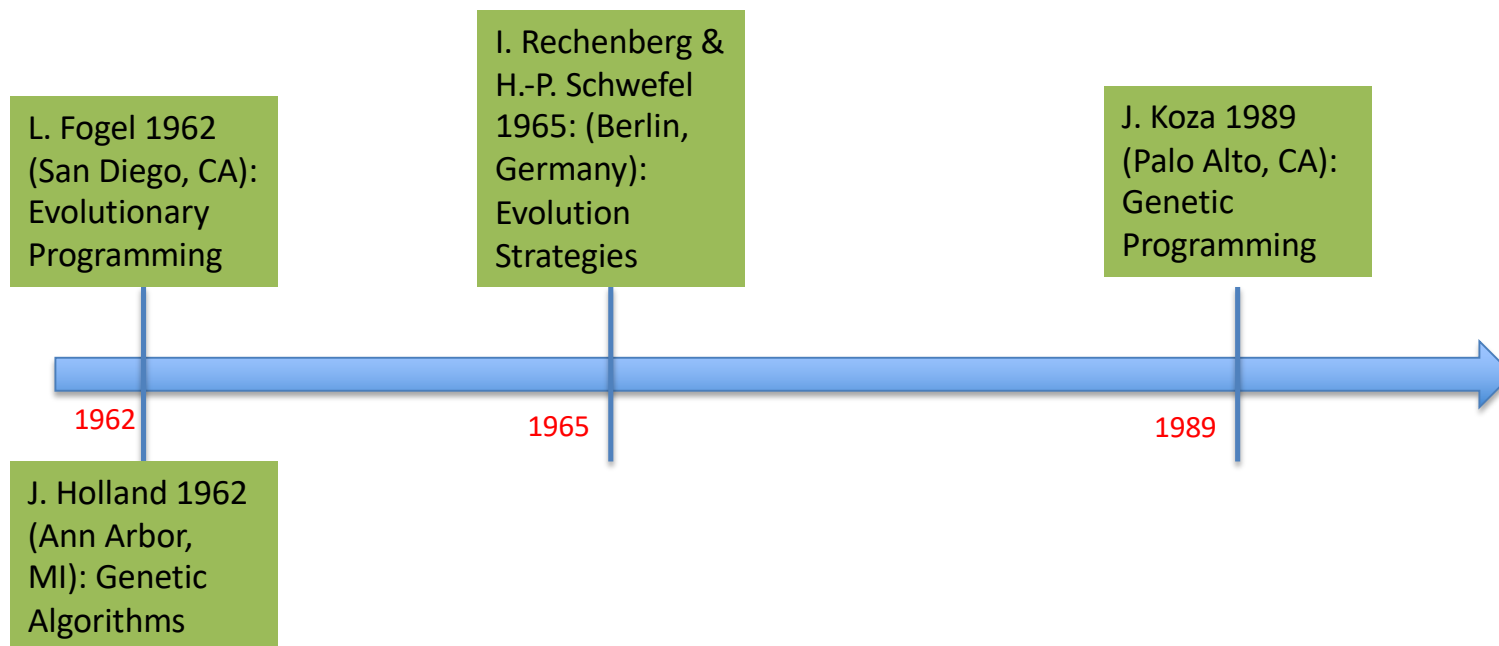
Taxonomy



Components of Evolutionary Computing

- Genetic Algorithms
 - Invented by John Holland (University of Michigan) in the 1960's
- Evolution Strategies
 - Invented by Ingo Rechenberg (Technical University Berlin) in the 1960's
- Started out as individual developments, but have begun to converge in the last few years

History



The Metaphor

Evolution

Problem Solving

Individual



Candidate Solution

Fitness Environment



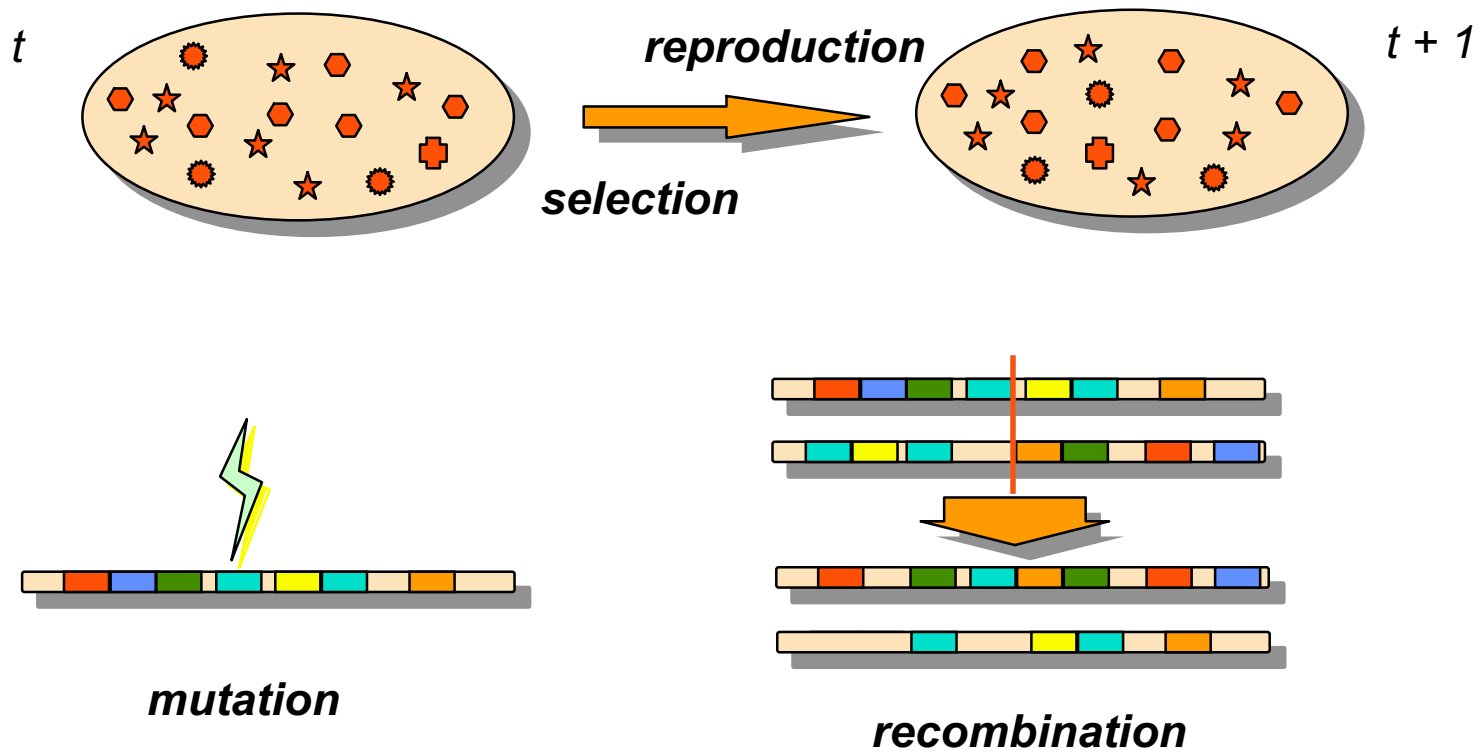
Quality

Environment



Problem

The Ingredients



Intelligence and Evolution

- One way of understanding intelligence is as capability of a creature to adapt its behavior to an ever-changing environment
- We normally think of adaptation as changes in the characteristics (including behaviors) of a single animal in response to experiences over its history
- But adaptation is also change over the characteristics of a species, over the generations in response to environmental change
- A creature must deal with other creatures of the same species who compete for resources, mates etc.
- There is also rivalry from other species which may be a direct (predator) or indirect (food, water, land, etc.) threat
- In nature, evolution operates on populations of organisms, ensuring by natural selection that characteristics that serve the members well tend to be passed on to the next generation, while those that didn't die out

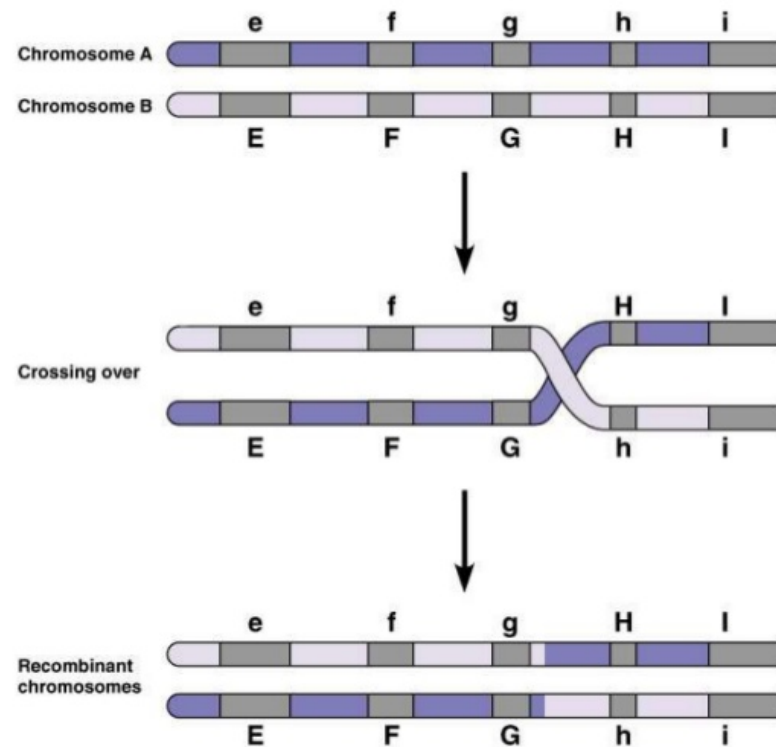
Evolution as Optimization

- Evolution can be seen as a process leading to the optimization of a population's ability to survive and thus reproduce in a specific environment. This ability is measured as evolutionary fitness
- This evolutionary fitness - the measure of the ability to respond adequately to the environment, is the quantity that is actually optimized in natural life
- Consider a normal population of rabbits. Some rabbits are faster than others. Any characteristic has a range of variation is natural, and is due to sexual reproduction and mutation
- We may say that the faster rabbits possess superior fitness, since they have a greater chance of avoiding foxes, surviving and then breeding
- If two parents have superior fitness, there is a good chance that a combination of their genes will produce an offspring with even higher fitness. We say there is crossover between the parents genes
- Over successive generations, the entire population of rabbits tends to become faster to meet their environment challenges in the face of foxes

Sexual Reproduction

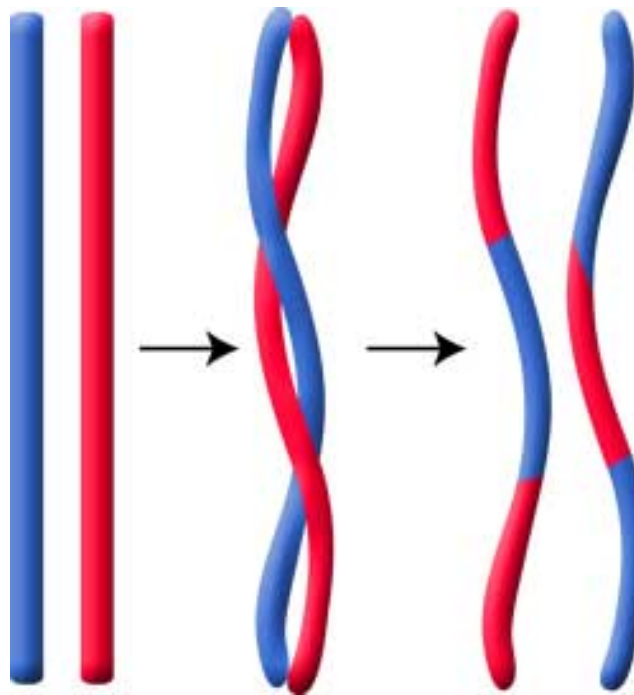
- The key to understanding evolution in nature lies in the basic biology of reproduction
- The chromosome is the basic carrier of the genes, which are the units of the genetic code that control an individual's characteristics. Each gene can take on one of a number of possible forms, called an allele
- An allele is like the value of a variable, and represents the effect that a gene will have on the physical makeup of a body
- An individual's particular sequence of alleles is called the genotype and it determines the expression of characteristics in the individual's body, called the phenotype
- In humans, most cells contain 23 pairs of chromosomes. But reproductive cells (sperm and ova) contain 23 single chromosomes, because they must merge with their opposite number to produce a new offspring
- During fertilization of the ova by the sperm, the chromosomes from each recombine to form the 23 pairs of the new individual

Sexual Reproduction - Single Crossing Over



Copyright © 2004 Pearson Education, Inc., publishing as Benjamin Cummings.

Sexual Reproduction - Single Crossing Over



Sexual Reproduction

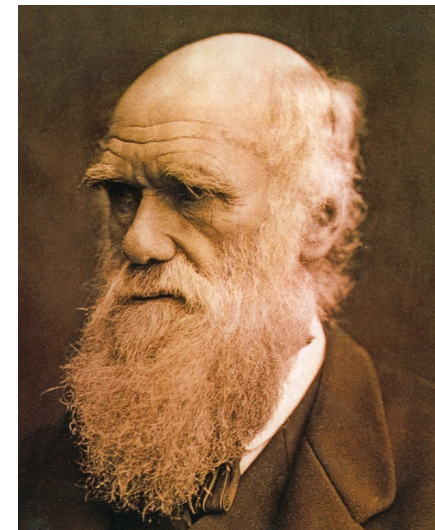
- Selection operates as survival and choice of mates between parents
- Recombination of genes is the mechanism that generates the next generation's characteristics
- Sometimes random copying errors, called mutations, occur during the recombination process. These are also important because they lead to new characteristics, usually useless, occasionally adaptive

Evolutionary Computation

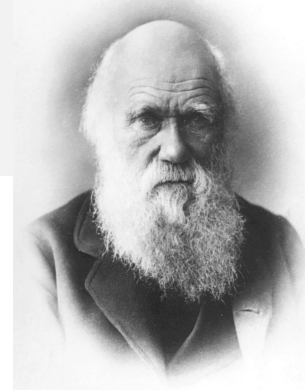
- Evolutionary computation simulates evolution on a computer. The result of such a simulation is a series of optimisation algorithms, usually based on a simple set of characteristics – the equivalent of genome
- Recall that optimisation iteratively improves the quality of solutions to some problem until an optimal (or at least feasible) solution is found
- Evolutionary computation is an umbrella term that includes genetic algorithms (Holland, 1975) , evolution strategies (Schwefel, 1981) and genetic programming (Koza, 1994)
- A-life researchers frequently experiment with populations of organisms put into artificial competition and subjected to the laws of natural selection

Darwinian Evolution

- Survival of the fittest
- Selection on phenotype
 - Through environment
- Genotypic inheritance
- Reproduction
- Blind variation



Darwinian Evolution



- Four Postulates
 1. Individuals within species are variable
 2. Some of the variations are passed on to offspring
 3. In every generation, more offspring are produced than can survive
 4. The survival and reproduction of individuals are not random:
 - The individuals who survive and go on to reproduce, or who reproduce the most, are those with the most favorable variations. They are naturally selected.

On the Origin of Species by Means of Natural Selection (Darwin 1859)

Nature of Natural Selection

“Based on "Evolutionary Analysis (Freeman & Herron, 2001)”

- Natural Evolution acts...
 - On Individuals, but the Consequences occur in the population
 - On Individuals, not groups
 - On Phenotypes, but evolution consist of changes in the Genotype
 - On existing traits, but can produce new traits
- Evolution...
 - Is backward looking
 - Is not perfect
 - Is nonrandom
 - Is not progressive

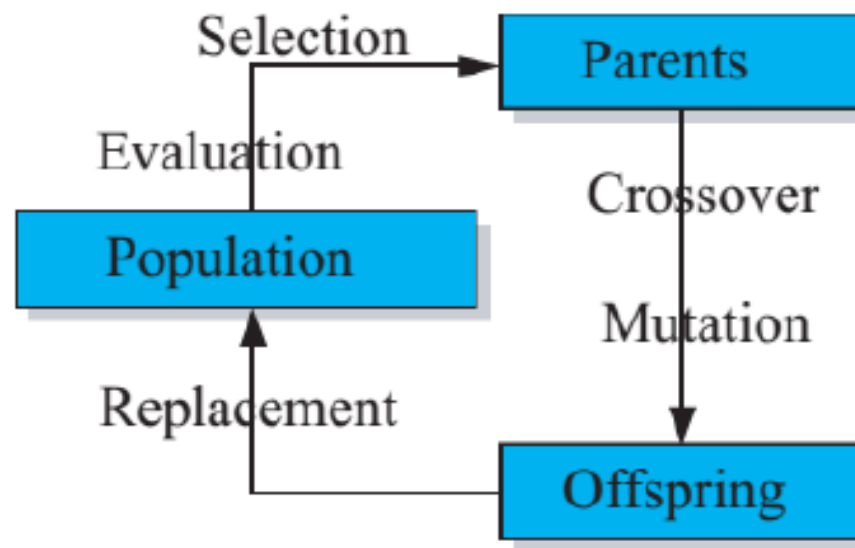
Evolutionary Algorithms

- Algorithms that are inspired by natural evolution
- Four Main Elements:
 - Group of Individuals - Population
 - Source of Variation - Genetic Operators
 - Reproductive Fitness - Fitness
 - Survival of the Fittest - Selection
- Search Process
 - Trial and Error
 - Recipe for choosing next trial

The Evolution Mechanism

- Increasing diversity by genetic operators
 - mutation
 - recombination
- Decreasing diversity by selection
 - of parents
 - of survivors

The Evolutionary Cycle



Domains of Application

- Numerical, Combinatorial Optimization
- System Modeling and Identification
- Planning and Control
- Engineering Design
- Data Mining
- Machine Learning
- Artificial Life

Performance

- Acceptable performance at acceptable costs on a wide range of problems
- Intrinsic parallelism (robustness, fault tolerance)
- Superior to other techniques on complex problems with:
 - lots of data, many free parameters
 - complex relationships between parameters
 - many (local) optima

Advantages

- No presumptions w.r.t. (with respect to) problem space
- Widely applicable
- Low development & application costs
- Easy to incorporate other methods
- Solutions are interpretable (unlike NN)
- Can be run interactively, accommodate user proposed solutions
- Provide many alternative solutions

Disadvantages

- No guarantee for optimal solution within finite time
- Weak theoretical basis
- May need parameter tuning
- Often computationally expensive, i.e. slow

The Argument

Evolution has optimized biological processes



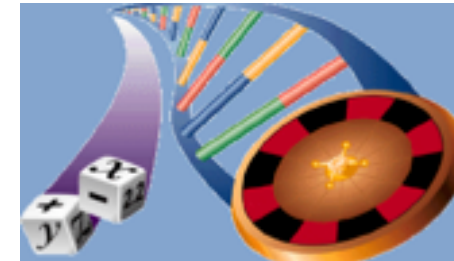
Therefore



Adoption of the evolutionary paradigm to computation and other problems can help us find optimal solutions.

The Concept of Natural Selection

- Limited number of resources
- Competition results in struggle for existence
- Success depends on fitness
 - fitness of an individual: how well-adapted an individual is to their environment. This is determined by their genes (blueprints for their physical and other characteristics).
- Successful individuals are able to reproduce and pass on their genes

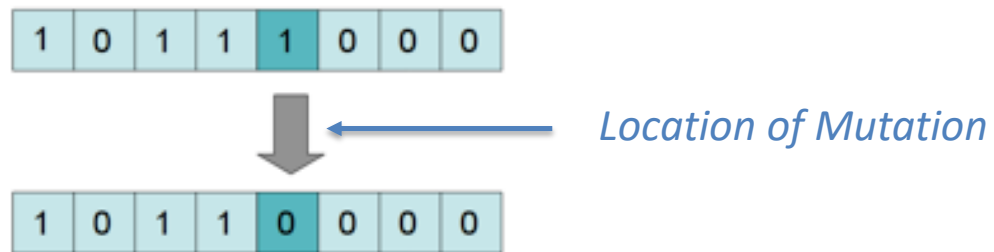


When changes occur ...

- Previously “fit” (well-adapted) individuals will no longer be best-suited for their environment
- Some members of the population will have genes that confer different characteristics than “the norm”. Some of these characteristics can make them more “fit” in the changing environment.

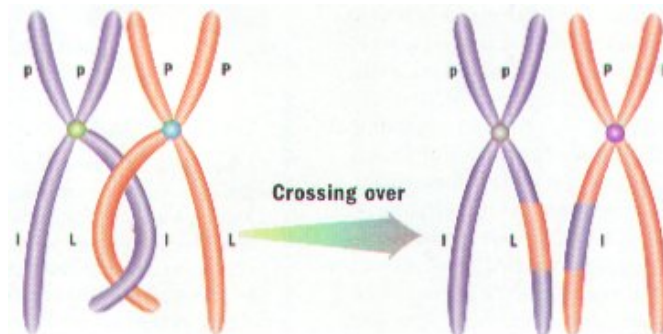
Major Agents of Genetic Change in Individuals

- Mutation in genes
 - may be due to various sources (e.g. UV rays, chemicals, etc.)



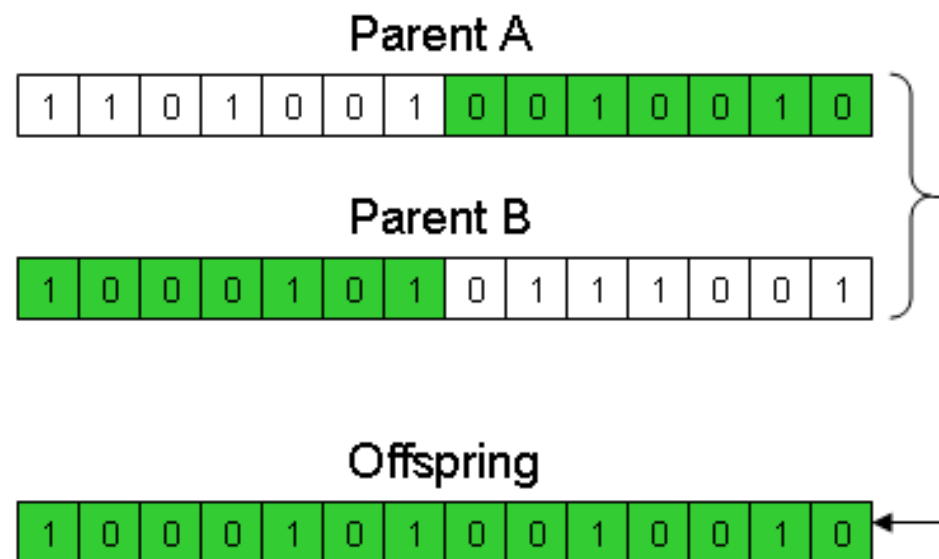
Major Agents of Genetic Change in Individuals

- Recombination (Crossing-Over)
 - occurs during reproduction - sections of genetic material exchanged between two chromosomes



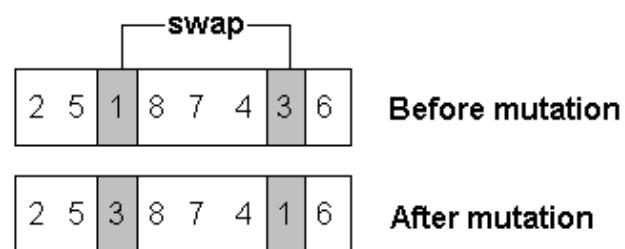
Genetic Operators

- Crossover



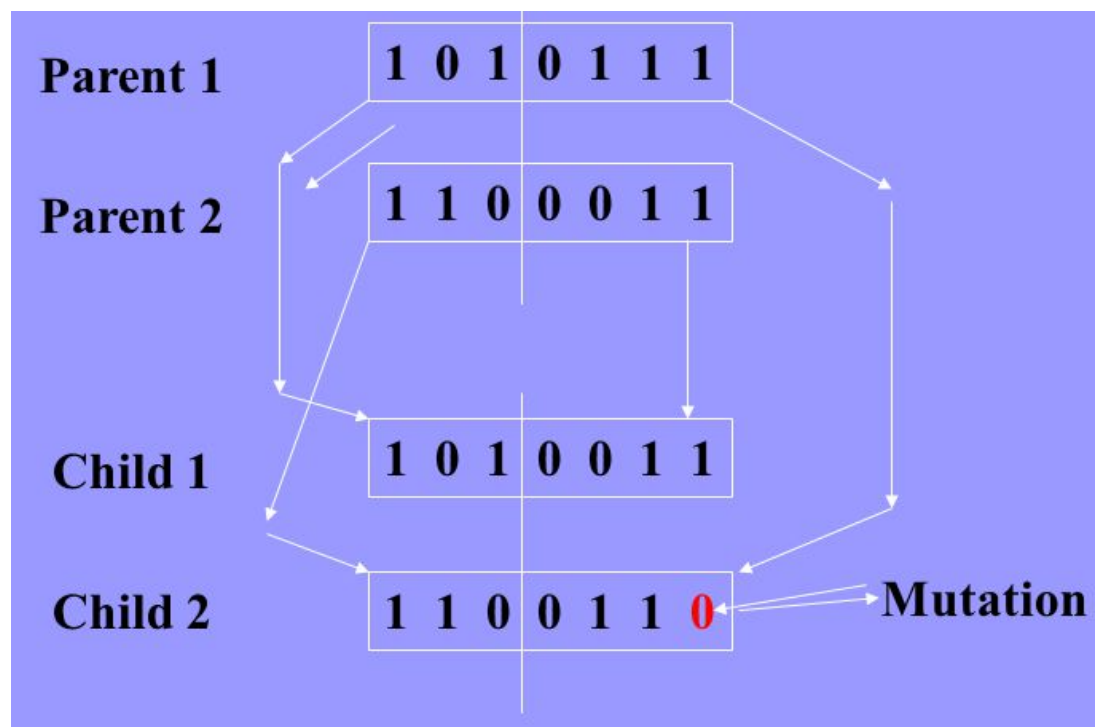
Genetic Operators

- Mutation



Production of New Chromosomes

- 2 parents give rise to 2 children



Question

Why use evolution as a model for solving computational problems?

The Nature of Computational Problems

- Require search through many possibilities to find a solution
 - (e.g. search through sets of rules for one set that best predicts the ups and downs of the financial markets)
 - Search space too big -- search won't return within our lifetimes
 - These types of problems are better solved using a parallel approach
- Require algorithm to be adaptive or to construct original solution
 - (e.g. interfaces that must adapt to idiosyncrasies of different users)

Why Evolution Proves to be a Good Model for Solving these Types of Problems

- Evolution is in effect a method of searching for the best (optimal) solution from a great number of possibilities
 - Possibilities - all individuals
 - Best solution - the most “fit” or well-adapted individual
- Evolution is a parallel process
 - Testing and changing of numerous species and individuals occur at the same time (or, in parallel)
- Evolution can be seen as a method that designs new (original) solutions to a changing environment

When to Use Evolutionary Computing Strategies

- When space to be searched is large
- When the “best” solution is not necessarily required
- Approach to solving a problem not well understood
- Problems with many parameters that need to be simultaneously optimized
- Problems that are difficult to describe mathematically

Genetic Algorithms

- Closely follows a biological approach to problem solving
- A simulated population of randomly selected individuals is generated then allowed to evolve

Basic Genetic Algorithm

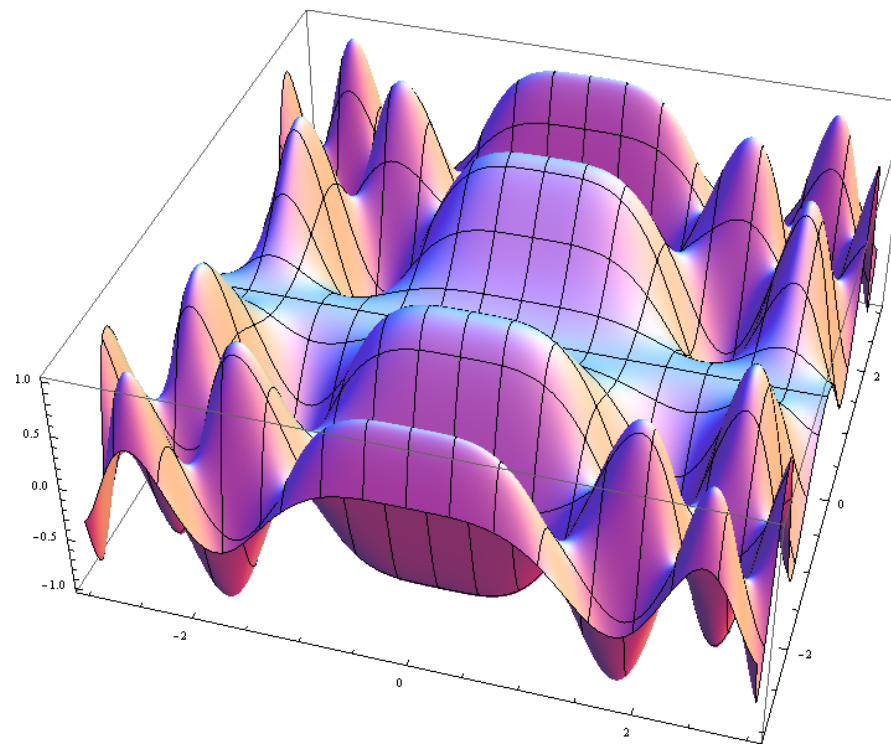
- Step 1. Generate a random population of n chromosomes
- Step 2. Assign a fitness to each individual
- Step 3. Repeat until n children have been produced
 - Choose 2 parents based on fitness proportional selection
 - Apply genetic operators to copies of the parents – Produce new chromosomes

Fitness Function

- For each individual in the population, evaluate its relative fitness
- For a problem with m parameters, the fitness can be plotted in an $m+1$ dimensional space

Sample Search Space

- A randomly generated population of individuals will be randomly distributed throughout the search space



Generations

- As each new generation of n individuals is generated, they replace their parent generation
- To achieve the desired results, 500 to 5000 generations are required

Ultimate Goal

- Each subsequent generation will evolve toward the global maximum
- After sufficient generations a near optimal solution will be present in the population of chromosomes

Dynamic Evolution

- Genetic algorithms can adapt to a dynamically changing search space
- Seek out the moving maximum via a parasitic fitness function
 - as the chromosomes adapt to the search space, so does the fitness function

Introduction to Genetic Algorithms

Genetic Algorithms Course (ALGEN)
MSc in Electrical and Computer Engineering

Filipe Azevedo
(fta@isep.ipp.pt)

History Of Genetic Algorithms

- “Evolutionary Computing” was introduced in the 1960s by **I. Rechenberg**.
- John Holland wrote the first book on Genetic Algorithms “**Adaptation in Natural and Artificial Systems**” in 1975.
- In 1992 **John Koza** used genetic algorithm to evolve programs to perform certain tasks. He called his method “**Genetic Programming**”.

Genetic Algorithms

- What are they?
 - Genetic Algorithms are **search** and **optimization** techniques based on Darwin's Principle of **Natural Selection**.
 - Evolutionary algorithms that make use of operations like mutation, recombination, and selection
- Uses?
 - Difficult search problems
 - Optimization problems
 - Machine learning
 - Adaptive rule-bases

Basic Idea Of Principle Of Natural Selection

“Select The Best, Discard The Rest”

Example of Natural Selection

- Giraffes have long necks.

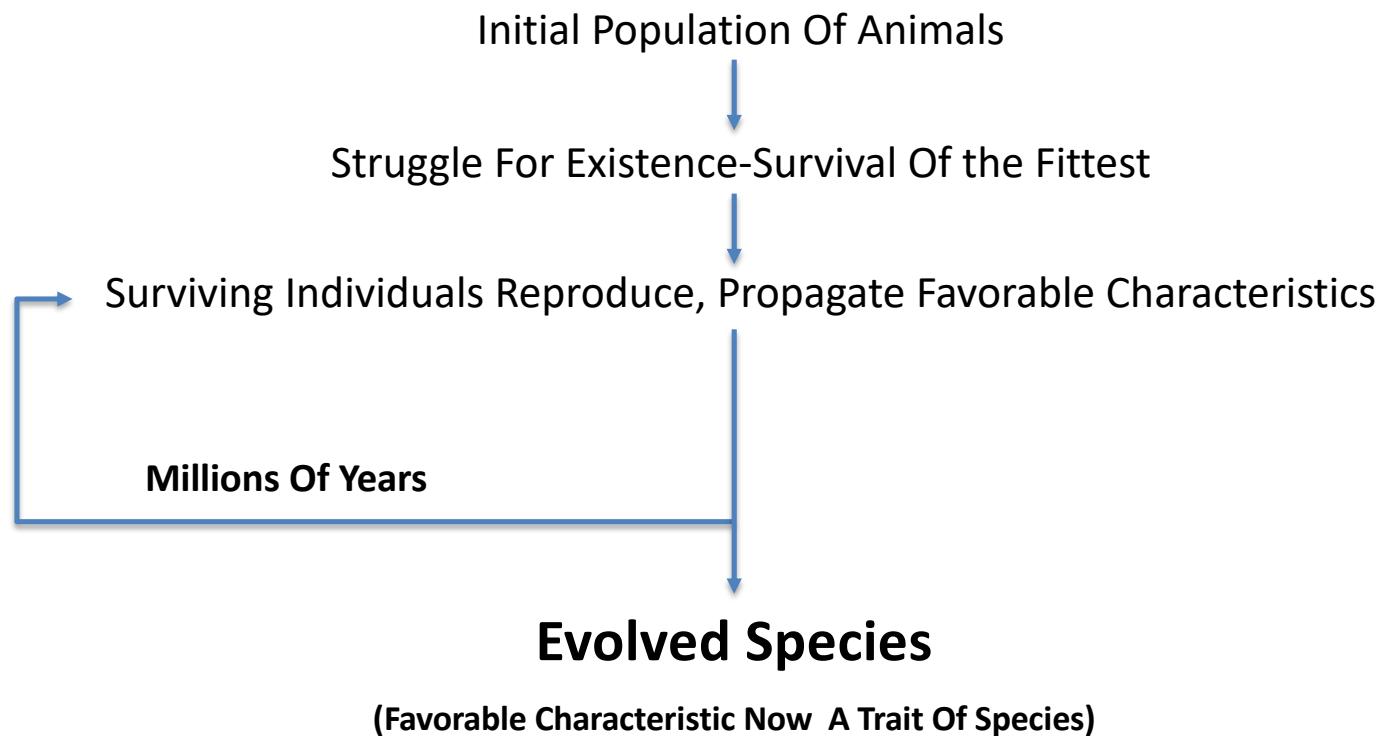
Giraffes with slightly longer necks could feed on leaves of higher branches when all lower ones had been eaten off.

- They had a better chance of survival.
- Favorable characteristic propagated through generations of giraffes.
- Now, evolved species has long necks.

NOTE: Longer necks may have been a deviant characteristic (**mutation**) initially but since it was favorable, was propagated over generations. Now an established trait.

So, some mutations are beneficial.

Evolution Through Natural Selection



Natural Selection

- Creatures that are not good at completing tasks like hunting have fewer chances of having offspring
- Creatures that are successful in completing basic tasks are more likely to transmit their attributes to the next generation since there will be more creatures born that can survive and pass on these attributes

Theory of Evolution

- Every organism has unique attributes that can be transmitted to its offspring
- Offspring are unique and have attributes from each parent
- Selective breeding can be used to manage changes from one generation to the next
- Nature applies certain pressures that cause individuals to evolve over time

Evolutionary Pressures

- Environment
 - Creatures must work to survive by finding resources like food and water
- Competition
 - Creatures within the same species compete with each other on similar tasks
 - Rivalry
 - Different species affect each other by direct confrontation (e.g. hunting) or indirectly by fighting for the same resources

Genetics

- Genome (class)
 - Sequence of genes describing the overall structure of the genetic for a particular species
- Genomics
 - Study of the meaning of the genes for a particular species
- Alleles
 - Values that can be assigned to a given gene
- Genotype (instance)
 - Sequence of alleles

Physical Properties

- Phenetics
 - Study of physical properties and morphology of creatures independent of genetic information
- Phenome
 - General structure of creatures body and attributes
- Phenotype
 - Particular instance of phenome realized as a unique creature
 - Product of genotype and environment forces

Conversions

- In real-world mapping between genotypes and phenotypes is hard
- In AI work it can be done by defining a convenient function or even designing encodings by hand
- It is often easier to adapt genetic operators to work with the evolutionary data structure used to represent the phenotype than to encode and decode phenotypes

Genetic Algorithmic Process

- Potential solution for problem domains are encoded using machine representation (e.g. bit strings) that supports variation and selection operations
- Mating and mutation operations produce new generation of solutions from parent encodings
- Fitness function judges the individuals that are “best” suited (e.g. most appropriate problem solution) for “survival”

Encoding

- The process of representing the solution in the form of a string that conveys the necessary information.
- Just as in a chromosome, each gene controls a particular characteristic of the individual, similarly, each bit in the string represents a characteristic of the solution.

Encoding Methods

- **Binary Encoding** – Most common method of encoding. Chromosomes are strings of 1s and 0s and each position in the chromosome represents a particular characteristic of the problem.

Chromosome A	10110010110011100101
Chromosome B	11111110000000011111

Encoding Methods

- **Binary Encoding**
- Example 1:
 - One variable function, say **0** to **15** numbers, numeric values, represented by 4 bit binary string.

Numeric value	4-bit string	Numeric value	4-bit string	Numeric value	4-bit string
0	0 0 0 0	6	0 1 1 0	12	1 1 0 0
1	0 0 0 1	7	0 1 1 1	13	1 1 0 1
2	0 0 1 0	8	1 0 0 0	14	1 1 1 0
3	0 0 1 1	9	1 0 0 1	15	1 1 1 1
4	0 1 0 0	10	1 0 1 0		
5	0 1 0 1	11	1 0 1 1		

Encoding Methods

- **Binary Encoding**
- Example 2:
 - Two variable function represented by 4 bit string for each variable.
 - Let two variables **X1** , **X2** as (**1011** and **0110**) .
 - Every variable will have both upper and lower limits as $X_i^L \leq X \leq X_i^U$.
 - Because 4-bit string can represent integers from 0 to 15, so (**0000 0000**) and (**1111 1111**) represent the points for **X₁** , **X₂** as (X_1^L , X_2^L) and (X_1^U , X_2^U), respectively.
 - Thus, an n-bit string can represent integers from **0** to $2^n - 1$, i.e. 2^n integers.

Encoding Methods

- Binary Encoding
- Example 2 (cont.)

Binary Coding

2	10	
2	5	0
2	2	1
	1	0

Remainder



Equivalent integer

1	0	1	0	
				$0 \times 2^0 = 0$
				$1 \times 2^1 = 2$
				$0 \times 2^2 = 0$
				$1 \times 2^3 = 8$
				10

Decoded binary substring

Let **X_i** be coded as a substring **S_i** of length **n_i** . Then decoded binary substring **S_i** is as

$$\sum_{k=0}^{K=n_i-1} 2^k S_k$$

where **S_i** can be **0** or **1** and the string **S** is represented as

$S_{n-1} \dots S_3 S_2 S_1 S_0$

Encoding Methods

- Binary Encoding
- Example: Decoding value
 - Consider a 4-bit string (0111)
 - The decoded value is equal to: $2^3 \times 0 + 2^2 \times 1 + 2^1 \times 1 + 2^0 \times 1 = 7$
 - Knowing X_i^L and X_i^U corresponding to (0000) and (1111), the equivalent value for any 4-bit string can be obtained as:

$$Xi = X_i^L + \frac{(X_i^U - X_i^L)}{(2^{ni} - 1)} \times (\text{decoded value of string})$$

Encoding Methods (cont.)

- **Permutation Encoding** – Useful in ordering problems such as the Traveling Salesman Problem (TSP). Example. In TSP, every chromosome is a string of numbers, each of which represents a city to be visited.

Chromosome A	1 5 3 2 6 4 7 9 8
Chromosome B	8 5 6 7 2 3 1 4 9

Encoding Methods (cont.)

- **Value Encoding** – Used in problems where complicated values, such as real numbers, are used and where binary encoding would not suffice.

Good for some problems, but **often necessary to develop some specific crossover and mutation techniques for these chromosomes.**

Chromosome A	1.235 5.323 0.454 2.321 2.454
Chromosome B	(left), (back), (left), (right), (forward)

Initialization

- Initial population must be a representative sample of the search space
- Random initialization can be a good idea (if the sample is large enough)
- Random number generator can not be biased
- Can reuse or seed population with existing genotypes based on algorithms or expert opinion or previous evolutionary cycles

Evaluation

- Each member of the population can be seen as candidate solution to a problem
- The fitness function determines the quality of each solution
- The fitness function takes a phenotype and returns a floating point number as its score
 - It is problem dependent so can be very simple
 - It can be a bottleneck if it is not carefully thought out (there are magic ways to create them)

Fitness Function

A fitness function quantifies the optimality of a solution (chromosome) so that that particular solution may be ranked against all the other solutions.

- A fitness value is assigned to each solution depending on how close it actually is to solving the problem.
- Ideal fitness function correlates closely to goal + quickly computable.
- Example. In TSP, $f(x)$ is sum of distances between the cities in solution. The lesser the value, the fitter the solution is.

Recombination

The process that determines which solutions are to be preserved and allowed to reproduce and which ones deserve to die out.

- The primary objective of the recombination operator is to **emphasize the good solutions** and **eliminate the bad solutions** in a population, **while keeping the population size constant**.
- “Selects The Best, Discards The Rest”.
- “Recombination” is different from “Reproduction”.

Recombination

- Identify the good solutions in a population.
- Make multiple copies of the good solutions.
- Eliminate bad solutions from the population so that multiple copies of good solutions can be placed in the population.

Roulette Wheel Selection

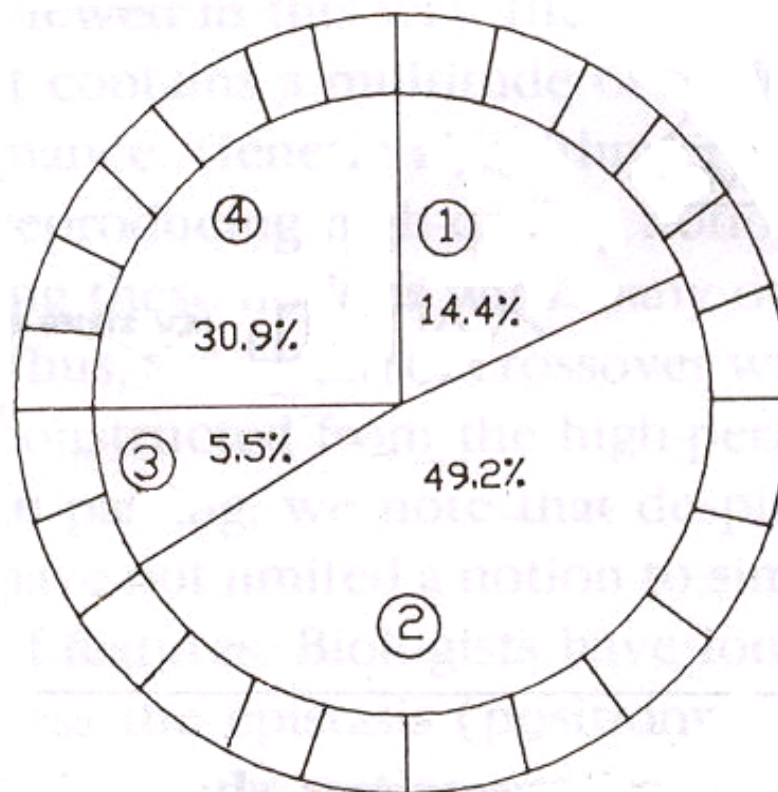
- Each current string in the population has a slot assigned to it which is in **proportion to its fitness**.
- We spin the weighted roulette wheel thus defined n times (where n is the total number of solutions).
- Each time Roulette Wheel stops, the string corresponding to that slot is created.

Strings that are fitter are assigned a larger slot and hence have a better chance of appearing in the new population.

Roulette Wheel Example (cont.)

No.	String	Fitness	% Of Total
1	01101	169	14.4
2	11000	576	49.2
3	01000	64	5.5
4	10011	361	30.9
Total		1170	100.0

Roulette Wheel Example (cont.)



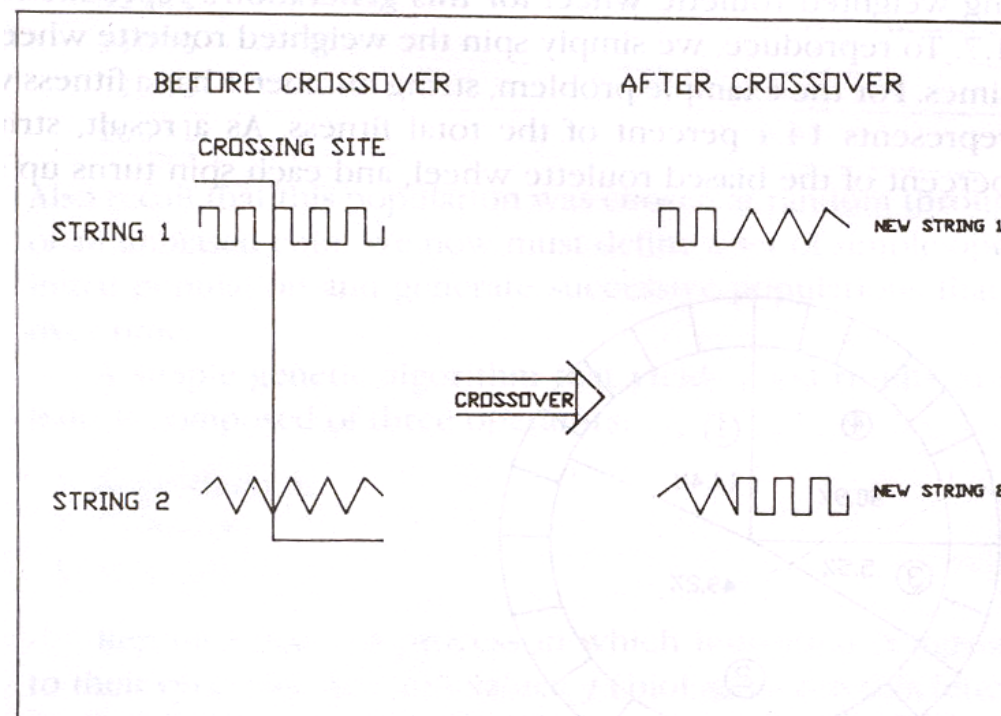
Crossover

It is the process in which two chromosomes (strings) combine their genetic material (bits) to produce a new offspring which possesses both their characteristics.

- Two strings are picked from the mating pool at random to cross over.
- The method chosen depends on the Encoding Method.

Crossover Methods

- **Single Point Crossover** - A random point is chosen on the individual chromosomes (strings) and the genetic material is exchanged at this point.



Crossover Methods (cont.)

- Single Point Crossover

Chromosome1	11011 00100110110
Chromosome 2	11011 11000011110
Offspring 1	11011 11000011110
Offspring 2	11011 00100110110

Crossover Methods (cont.)

- **Two-Point Crossover** - Two random points are chosen on the individual chromosomes (strings) and the genetic material is exchanged at these points.

Chromosome1	11011 00100 110110
Chromosome 2	10101 11000 011110
Offspring 1	10101 00100 011110
Offspring 2	11011 11000 110110

NOTE: These chromosomes are different from the last example.

Crossover Methods (cont.)

- **Uniform Crossover** - Each gene (bit) is selected randomly from one of the corresponding genes of the parent chromosomes.

Chromosome1	11011 00100 110110
Chromosome 2	10101 11000 011110
Offspring	10111 00000 110110

NOTE: Uniform Crossover yields ONLY 1 offspring.

Crossover

- Crossover between 2 good solutions **MAY NOT ALWAYS** yield a better or as good a solution.
- Since parents are good, probability of the child being good is high.
- If offspring is not good (poor solution), it will be removed in the next iteration during “Selection”.

Elitism

Elitism is a method which copies the best chromosome to the new offspring population before crossover and mutation.

- When creating a new population by crossover or mutation the best chromosome might be lost.
- Forces GAs to retain some number of the best individuals at each generation.
- Has been found that elitism significantly improves performance.

Mutation

It is the process by which a string is deliberately changed so as to maintain diversity in the population set.

- We saw in the giraffes' example, that mutations could be beneficial.
- **Mutation Probability**- determines how often the parts of a chromosome will be mutated.

Example of Mutation

- For chromosomes using Binary Encoding, randomly selected bits are inverted.

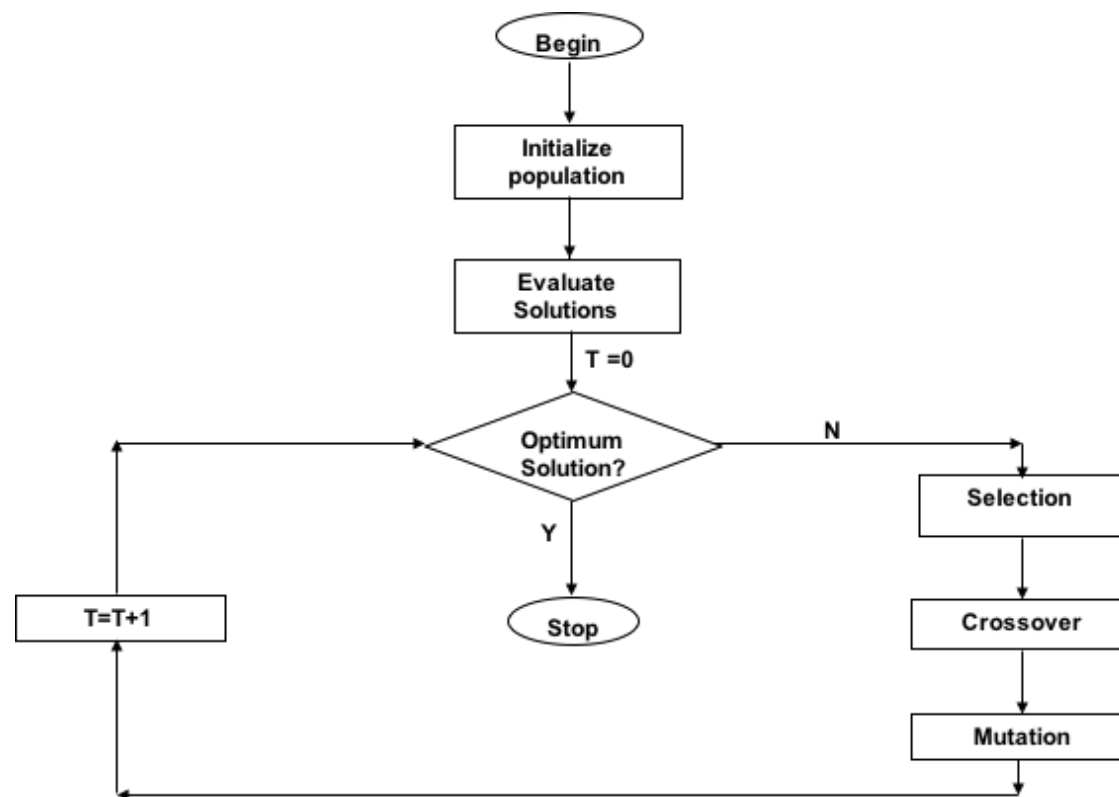
Offspring	1101 1 00100 1 1 0110
Mutated Offspring	1101 0 00100 1 0 0110

NOTE: The number of bits to be inverted depends on the Mutation Probability.

Nature to Computer Mapping

Nature	Computer
Population	Set of solutions.
Individual	Solution to a problem.
Fitness	Quality of a solution.
Chromosome	Encoding for a Solution.
Gene	Part of the encoding of a solution.
Reproduction	Crossover

Mechanism Of GAs



Why use genetic algorithms?

- They can solve hard problems
- Easy to interface genetic algorithms to existing simulations and models
- GA's are extensible
- GA's are easy to hybridize
- GA's work by sampling, so populations can be sized to detect differences with specified error rates
- Use little problem specific code

Advantages Of GAs

- **Global Search Methods:** GAs search for the function optimum starting from a population of points of the function domain, not a single one. This characteristic suggests that GAs are global search methods. They can, in fact, climb many peaks in parallel, reducing the probability of finding local minima, which is one of the drawbacks of traditional optimization methods.
- **Blind Search Methods:** GAs only use the information about the objective function. They do not require knowledge of the first derivative or any other auxiliary information, allowing a number of problems to be solved without the need to formulate restrictive assumptions. For this reason, GAs are often called blind search methods.
- GAs use probabilistic transition rules during iterations, unlike the traditional methods that use fixed transition rules.
 - This makes them more **robust** and applicable to a large range of problems.

Advantages Of GAs (cont.)

- **GAs can be easily used in parallel machines** - Since in real-world design optimization problems, most computational time is spent in evaluating a solution, with multiple processors all solutions in a population can be evaluated in a distributed manner. This reduces the overall computational time substantially.

Traveling Salesman Problem

- The Traveling Salesman Problem is defined as:

“It is given a set of cities and a symmetric distance matrix that indicates the cost of travel from each city to every other city.

The goal is to find the shortest circular tour, visiting every city exactly once, so as to minimize the total travel cost, which includes the cost of traveling from the last city back to the first city”.

Encoding

- Every city is represented with an integer .
- Consider 6 Indian cities – Mumbai, Nagpur , Calcutta, Delhi , Bangalore and Chennai and assign a number to each.

Mumbai	1
Nagpur	2
Calcutta	3
Delhi	4
Bangalore	5
Chennai	6

Encoding (cont.)

- Thus a path would be represented as a sequence of integers from 1 to 6.
- The path [1 2 3 4 5 6] represents a path from Mumbai to Nagpur, Nagpur to Calcutta, Calcutta to Delhi, Delhi to Bangalore, Bangalore to Chennai, and finally from Chennai to Mumbai.
- This is an example of Permutation Encoding as the position of the elements determines the fitness of the solution.

Fitness Function

- The fitness function will be the **total cost of the tour** represented by each chromosome.
- This can be calculated as the **sum of the distances** traversed in each travel segment.
- The **Lesser The Sum, The Fitter The Solution** Represented By That Chromosome.

Distance/Cost Matrix For TSP

	1	2	3	4	5	6
1	0	863	1987	1407	998	1369
2	863	0	1124	1012	1049	1083
3	1987	1124	0	1461	1881	1676
4	1407	1012	1461	0	2061	2095
5	998	1049	1881	2061	0	331
6	1369	1083	1676	2095	331	0

Cost matrix for six city example.

Distances in Kilometers

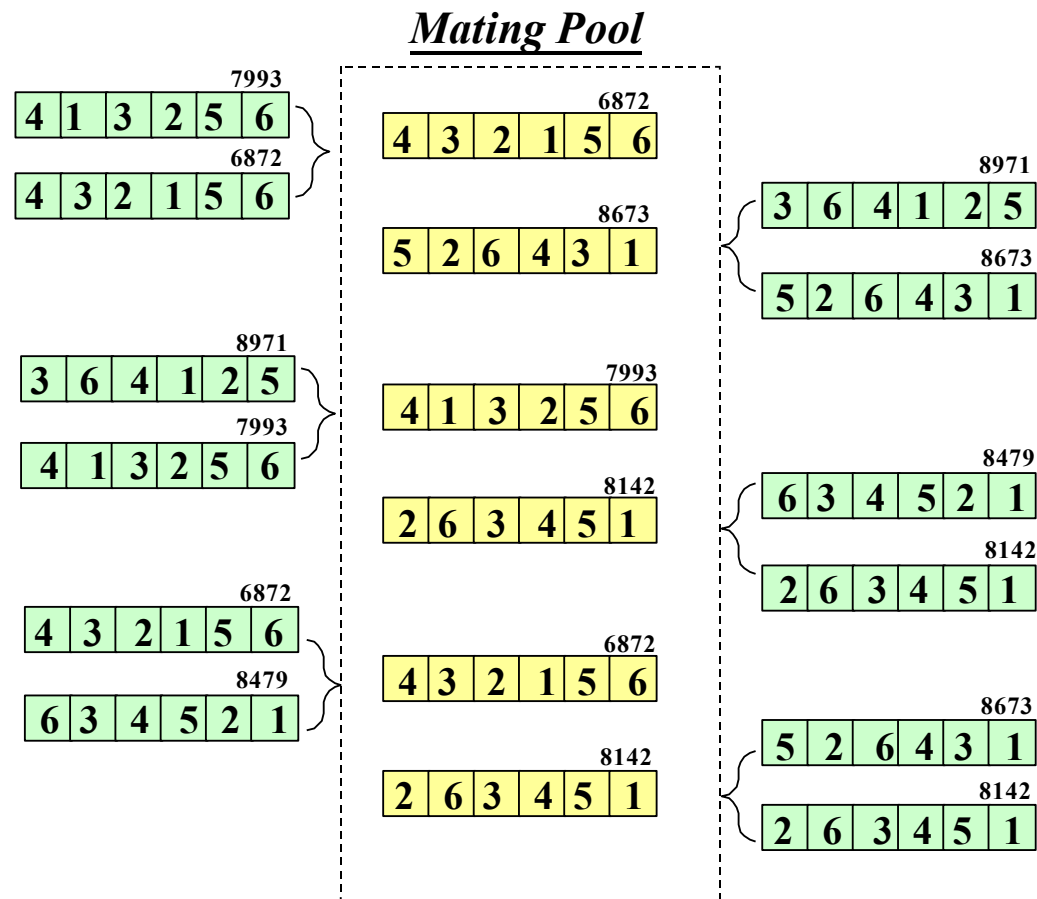
Fitness Function (cont.)

- So, for a chromosome [4 1 3 2 5 6], the total cost of travel or fitness will be calculated as shown below
- $$\begin{aligned}\text{Fitness} &= 1407 + 1987 + 1124 + 1049 + 331 + 2095 \\ &= 7993 \text{ kms.}\end{aligned}$$
- Since the objective is to **Minimize** the distance, the lesser the total distance, the fitter the solution.

Selection Operator

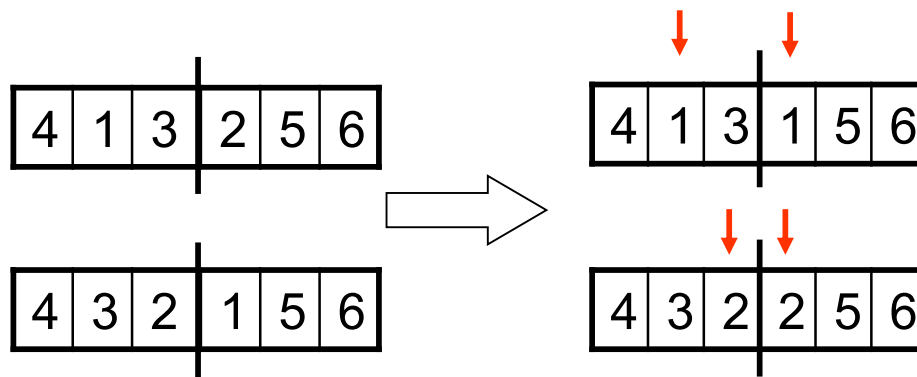
- Was used **Tournament Selection**.
- As the name suggests **tournaments** are played between two solutions and the better solution is chosen and placed in the **mating pool**.
- Two other solutions are picked again and another slot in the **mating pool** is filled up with the better solution.

Tournament Selection



Why cannot be used single-point crossover:

- Single point crossover method randomly selects a crossover point in the string and swaps the substrings.
- This may produce some **invalid offsprings** as shown below.



Crossover Operator

- Was used the **Enhanced Edge Recombination** operator (T.Starkweather, et al, 'A Comparison of Genetic Sequencing Operators, International Conference of GAs, 1991).
- This operator is different from other genetic sequencing operators in that it emphasizes **adjacency information** instead of the order or position of items in the sequence.
- The algorithm for the Edge-Recombination Operator involves constructing an Edge Table first.

Edge Table

- The Edge Table is an adjacency table that lists links into and out of a city found in the two parent sequences.
- If an item is already in the edge table and we try to insert it again, that element of a sequence must be a common edge and is represented by inverting its sign.

Finding The Edge Table

Parent 1

4	1	3	2	5	6
---	---	---	---	---	---

Parent 2

4	3	2	1	5	6
---	---	---	---	---	---

1	4	3	2	5
2	-3	5	1	
3	1	-2	4	
4	-6	1	3	
5	2	-6	1	
6	-5	-4		

Enhanced Edge Recombination Algorithm

1. Choose the initial city from one of the two parent tours. (It can be chosen randomly as according to criteria outlined in step 4). This is the current city.
2. Remove all occurrences of the current city from the left hand side of the edge table.(These can be found by referring to the edge-list for the current city).
3. If the current city has entries in it's edge-list, go to step 4 otherwise go to step 5.
4. Find which of the cities in the edge-list of the current city has the fewest entries in it's own edge-list. The city with fewest entries becomes the current city. In case a negative integer is present, it is given preference. Ties are broken randomly. Go to step 2.
5. If there are no remaining unvisited cities, then stop. Otherwise, randomly choose an unvisited city and go to step 2.

Example Of Enhanced Edge Recombination Operator

Step 1

1	4	3	2	5
2	-3	5	1	
3	1	-2	4	
4	-6	1	3	
5	2	-6	1	
6	-5	-4		

↓

4					
---	--	--	--	--	--

Step 2

1	3	2	5
2	-3	5	1
3	1	-2	
4	-6	1	3
5	2	-6	1
6	-5		

↓

4	6				
---	---	--	--	--	--

Example Of Enhanced Edge Recombination Operator (cont.)

Step 3

1	3	2	5	
2	-3	5	1	
3	1	-2		
4	1	3		
5	2	1		
6	-5			

↓

4	6	5			
---	---	---	--	--	--

Step 4

1	3	2
2	-3	1
3	1	-2
4	1	3
5	2	1
6		

↓

4	6	5	1		
---	---	---	---	--	--

Example Of Enhanced Edge Recombination Operator (cont.)

Step 5

1	3	2
2	-3	
3	-2	
4	3	
5	2	
6		

↓

4	6	5	1	3	
---	---	---	---	---	--

Step 6

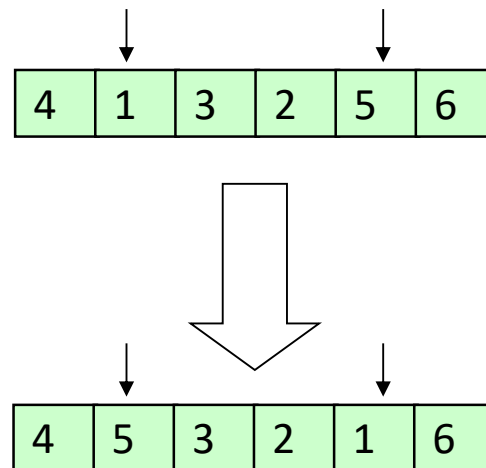
1	2
2	
3	-2
4	
5	2
6	

↓

4	6	5	1	3	2
---	---	---	---	---	---

Mutation Operator

1. The mutation operator induces a change in the solution, so as to maintain diversity in the population and prevent **Premature Convergence**.
2. The string was mutated by randomly selecting any two cities and interchanging their positions in the solution, thus giving rise to a new tour.



Input To Program

Travelling Salesman Problem using Genetic Algorithm

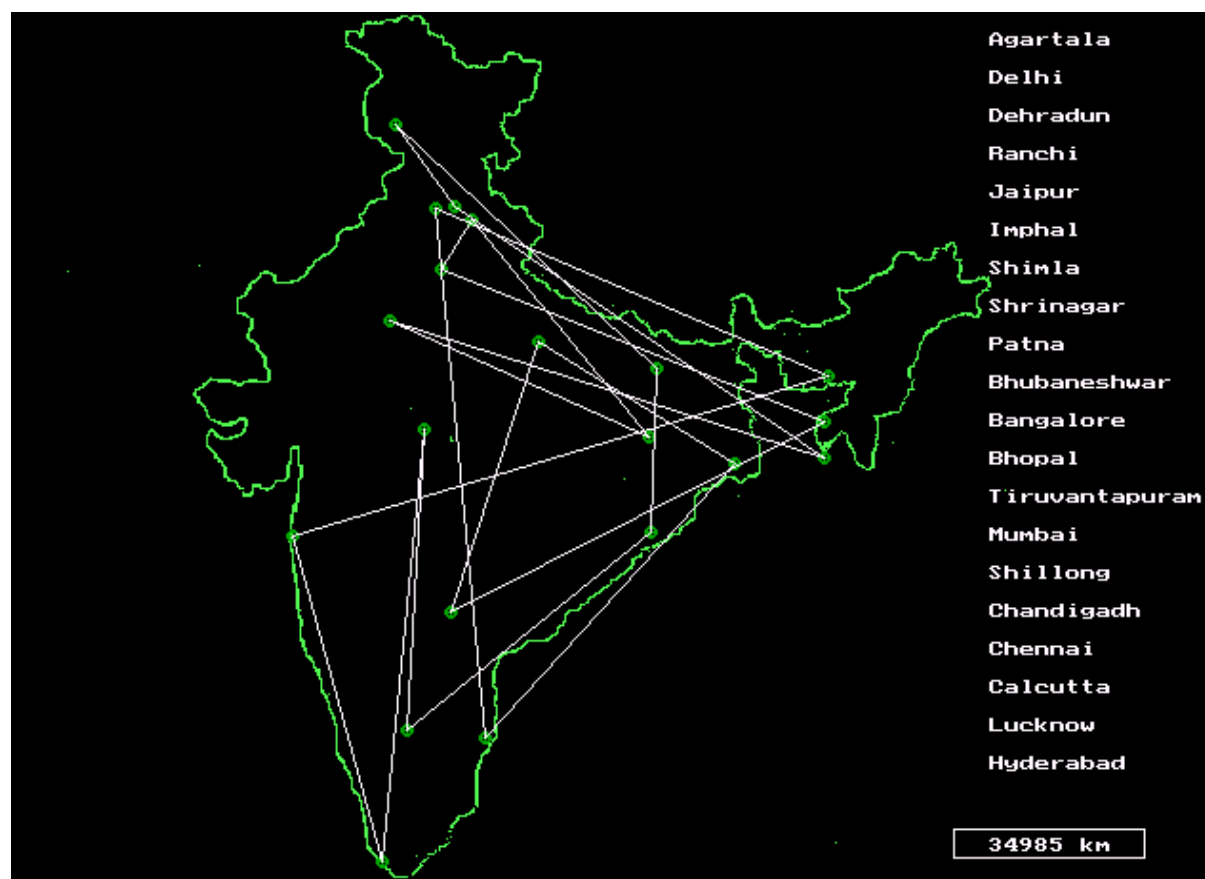
Enter population size : 20

Enter number of cities : 20

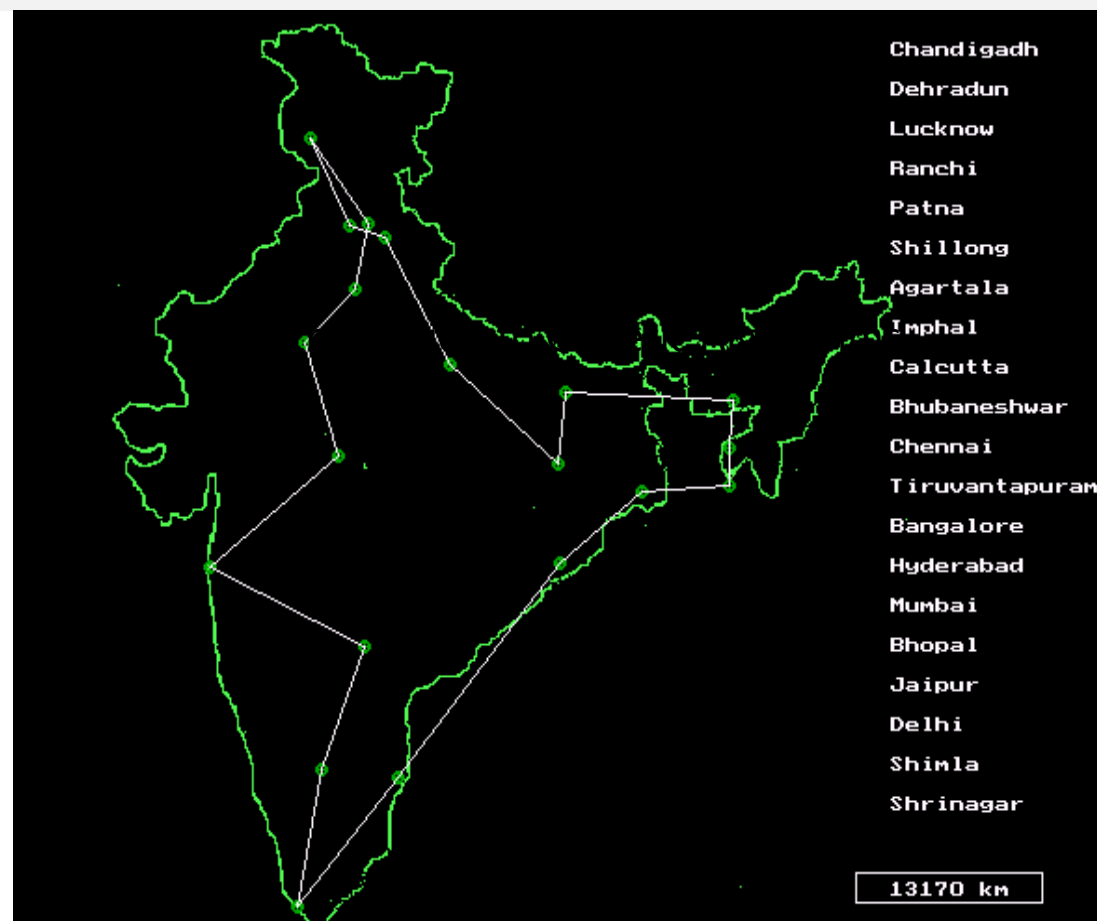
Enter maximum generations : 10000

Enter mutation probability : 0.09

Initial Output For 20 cities : Distance=34985 km
Initial Population



**Final Output For 20 cities : Distance=13170 km
Generation 4786**



Swarm Intelligence

Genetic Algorithms Course (ALGEN)
MSc in Electrical and Computer Engineering

Filipe Azevedo
(fta@isep.ipp.pt)

Particle Swarm Optimization (PSO)

Genetic Algorithms Course (ALGEN)
MSc in Electrical and Computer Engineering

Filipe Azevedo
(fta@isep.ipp.pt)

Origins and Inspiration from Natural Systems

- Developed by **Jim Kennedy**, Bureau of Labor Statistics, U.S. Department of Labor and **Russ Eberhart**, Purdue University at 1995
- A concept for optimizing nonlinear functions using particle swarm methodology

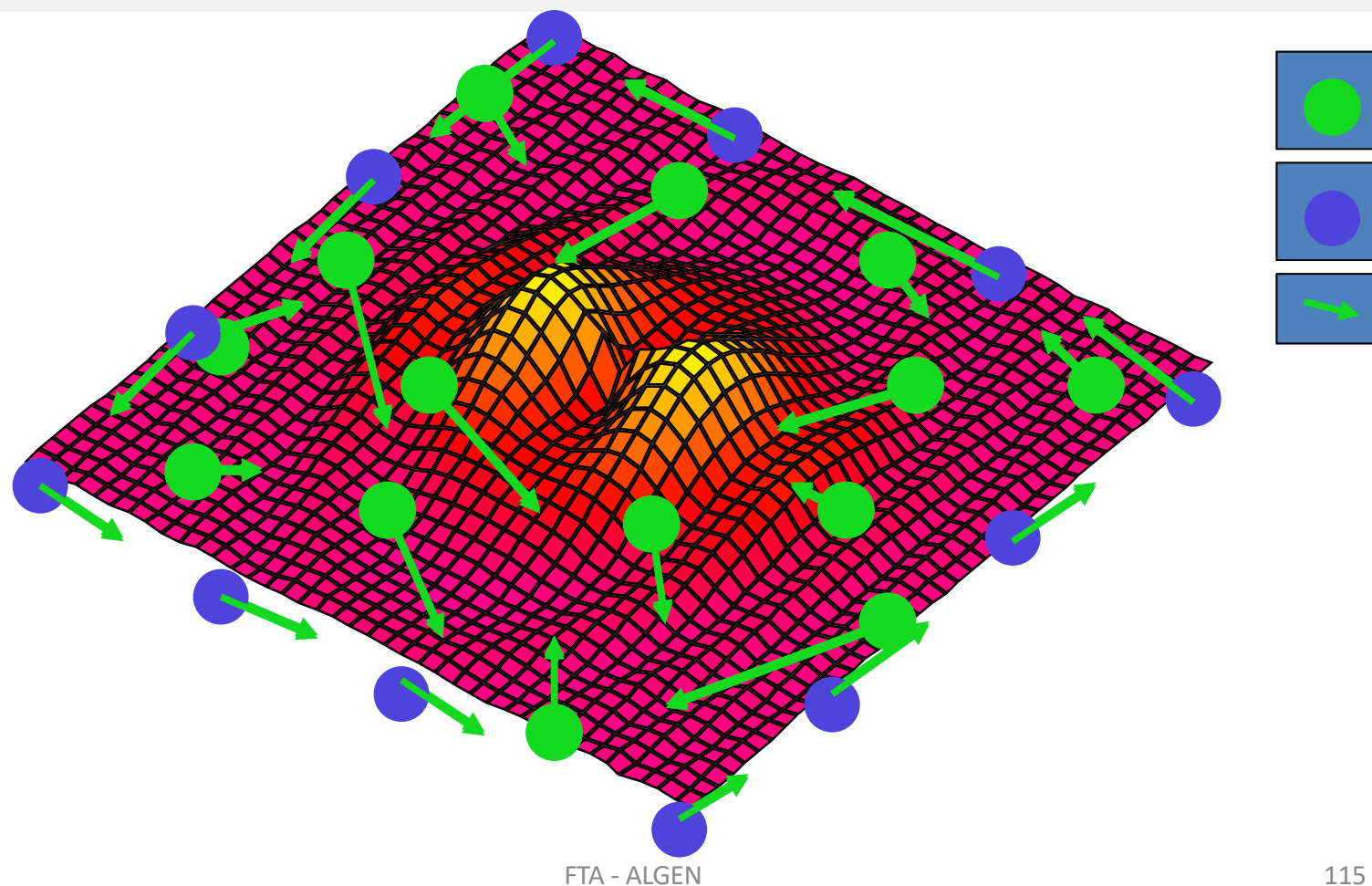
The basic idea

- Each particle is searching for the optimum.
- Each particle is moving and hence has a velocity.
- Each particle remembers the position it was in where it had its best result so far (its personal best).
- But this would not be much good on its own; particles need help in figuring out where to search.

The basic idea

- The particles in the swarm co-operate. They exchange information about what they've discovered in the places they have visited
- The co-operation is very simple. In basic PSO it is like this:
 - A particle has a neighborhood associated with it.
 - A particle knows the fitnesses of those in its neighborhood, and uses the position of the one with best fitness.
 - This position is simply used to adjust the particle's velocity

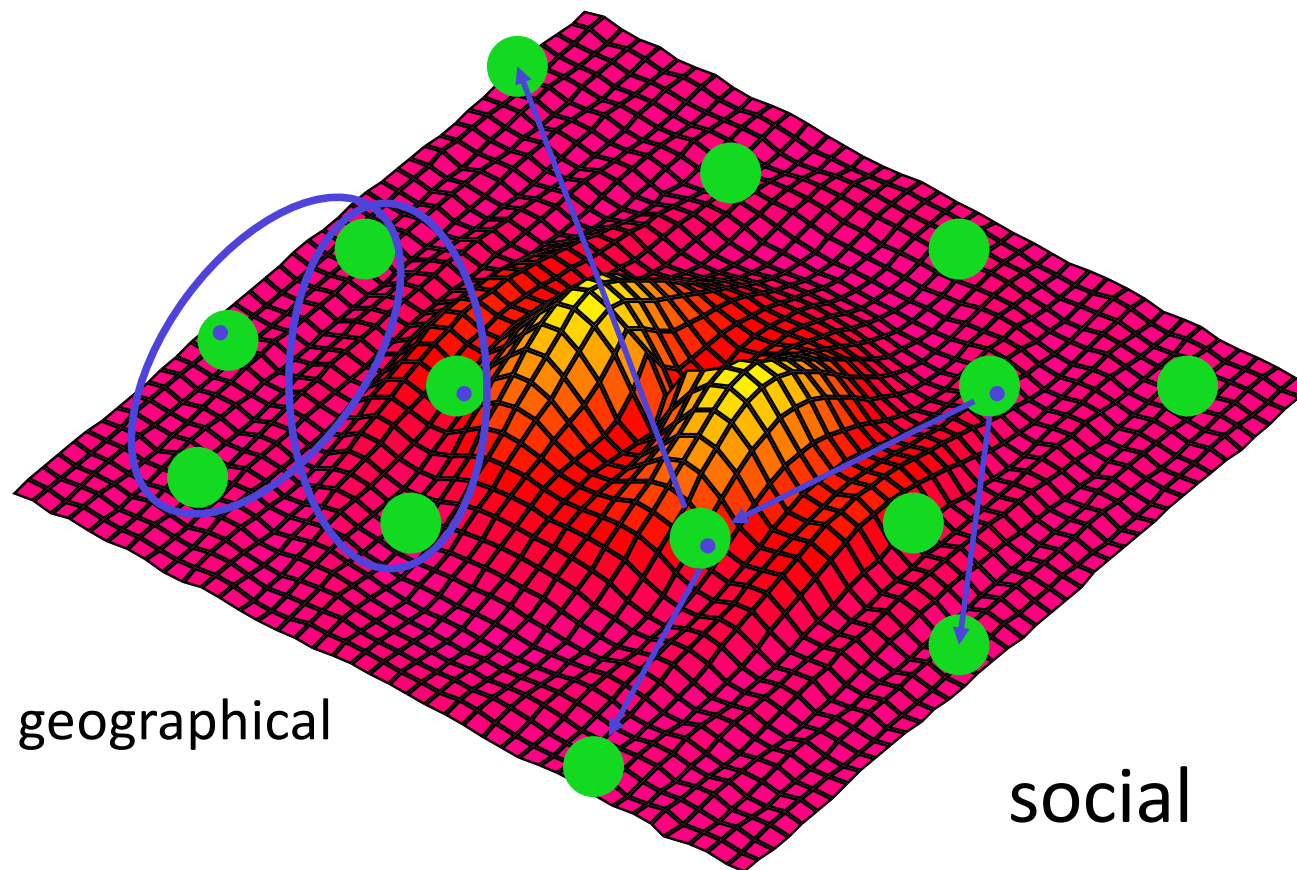
Initialization. Positions and velocities



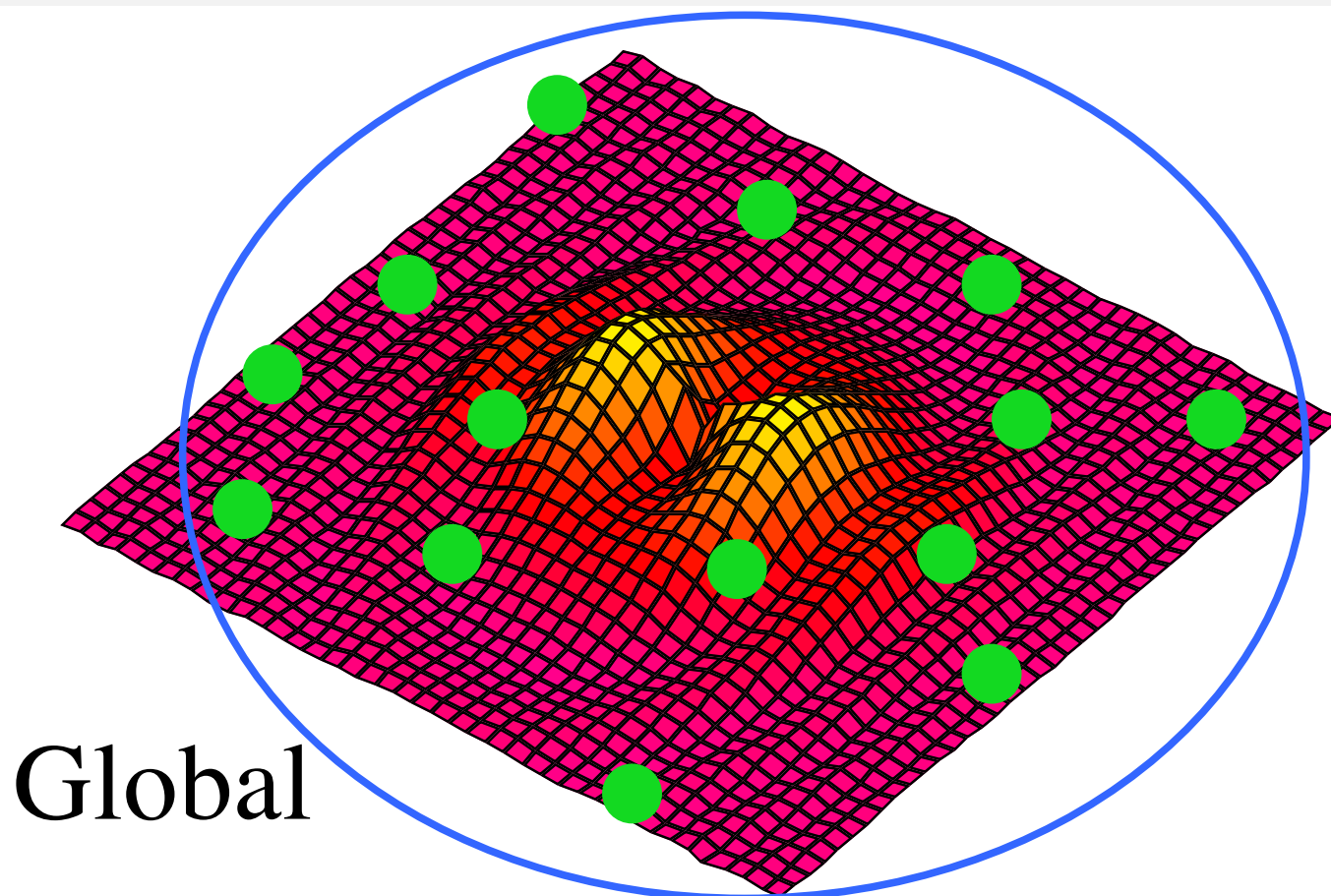
What a particle does

- In each timestep, a particle has to move to a new position. It does this by adjusting its velocity.
 - The adjustment is essentially this:
 - The current velocity PLUS
 - A weighted random portion in the direction of its personal best PLUS
 - A weighted random portion in the direction of the neighborhood best.
- Having worked out a new velocity, its position is simply its old position plus the new velocity.

Neighborhoods



Neighborhoods

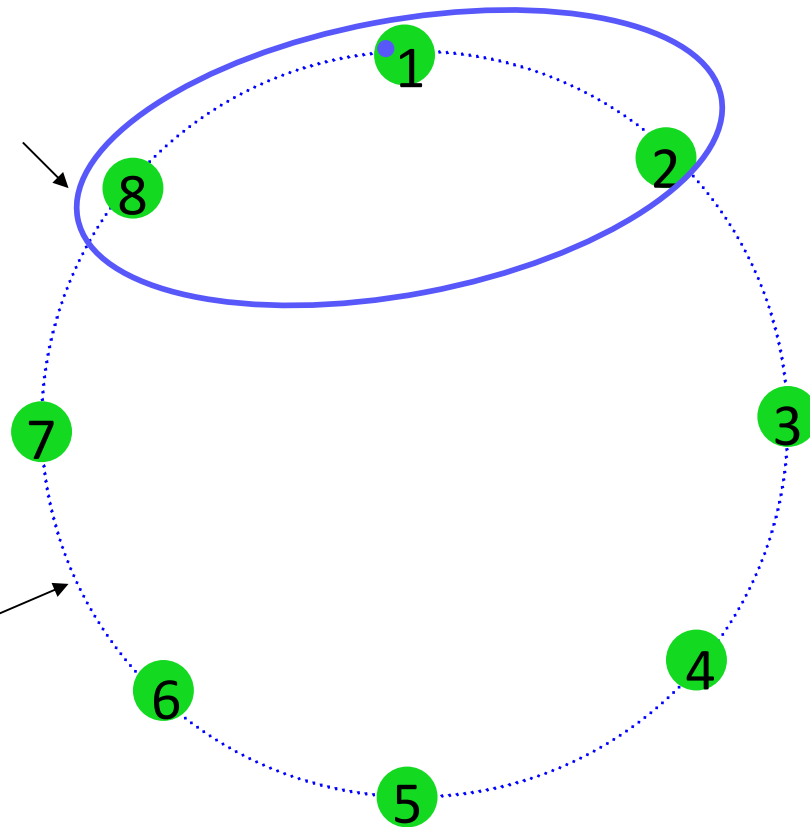


Global

The circular neighborhood

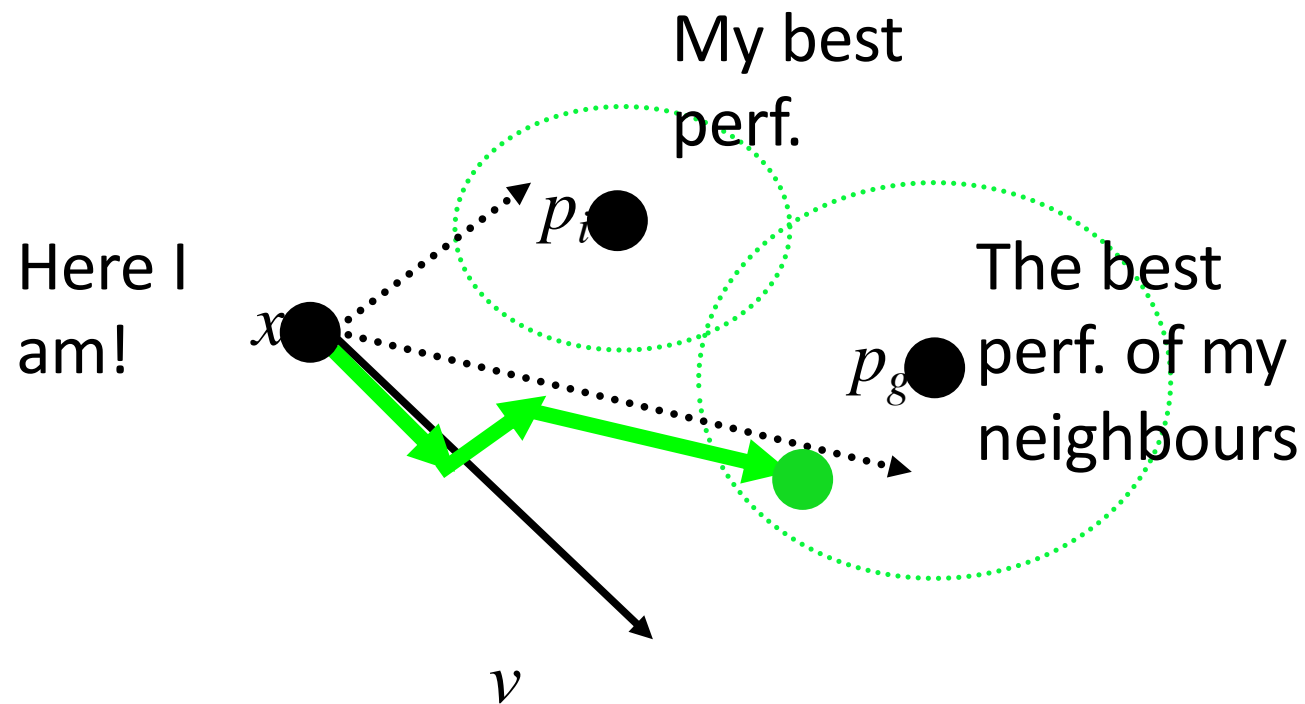
Particle 1's 3-
neighbourhood

Virtual circle



Particles Positions

- Particles Adjust their positions according to a “Psychosocial compromise” between what an individual is comfortable with, and what society reckons



Algorithm

1. Create a 'population' of agents (particles) uniformly distributed over X
2. Evaluate each particle's position according to the objective function
3. If a particle's current position is better than its previous best position, update it
4. Determine the best particle (according to the particle's previous best positions)

Algorithm

5. Update particles' velocities:

$$\mathbf{v}_i^{t+1} = \underbrace{\mathbf{v}_i^t}_{\text{inertia}} + \underbrace{c_1 \mathbf{U}_1^t (\mathbf{pb}_i^t - \mathbf{p}_i^t)}_{\text{personal influence}} + \underbrace{c_2 \mathbf{U}_2^t (\mathbf{gb}^t - \mathbf{p}_i^t)}_{\text{social influence}}$$

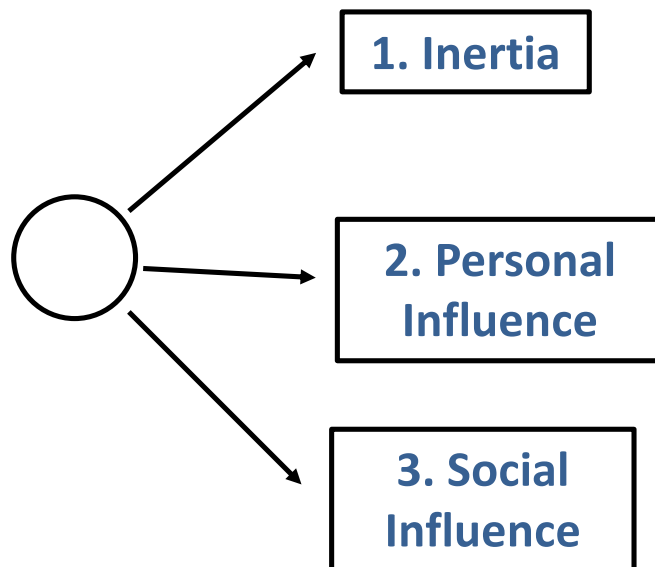
6. Move particles to their new positions:

$$\mathbf{p}_i^{t+1} = \mathbf{p}_i^t + \mathbf{v}_i^{t+1}$$

7. Go to step 2 until stopping criteria are satisfied

Particle's velocity

$$\mathbf{v}_i^{t+1} = \underbrace{\mathbf{v}_i^t}_{\text{inertia}} + \underbrace{c_1 \mathbf{U}_1^t (\mathbf{pb}_i^t - \mathbf{p}_i^t)}_{\text{personal influence}} + \underbrace{c_2 \mathbf{U}_2^t (\mathbf{gb}^t - \mathbf{p}_i^t)}_{\text{social influence}}$$



- Makes the particle move in the same direction and with the same velocity
- Improves the individual
- Makes the particle return to a previous position, better than the current
- Conservative
- Makes the particle follow the best neighbors direction

Algorithm

- Intensification: explores the previous solutions, finds the best solution of a given region
- Diversification: searches new solutions, finds the regions with potentially the best solutions
- In PSO:

$$\mathbf{v}_i^{t+1} = \underbrace{\mathbf{v}_i^t}_{\text{Diversification}} + \underbrace{\mathbf{c}_1 \mathbf{U}_1^t (\mathbf{pb}_i^t - \mathbf{p}_i^t) + \mathbf{c}_2 \mathbf{U}_2^t (\mathbf{gb}^t - \mathbf{p}_i^t)}_{\text{Intensification}}$$

Advantages and Disadvantages

- **Advantages**
 - Insensitive to scaling of design variables
 - Simple implementation
 - Easily parallelized for concurrent processing
 - Derivative free
 - Very few algorithm parameters
 - Very efficient global search algorithm
- **Disadvantages**
 - Tendency to a fast and premature convergence in mid optimum points
 - Slow convergence in refined search stage (weak local search ability)

Different Approaches

- **Several approaches**
 - 2-D Otsu PSO
 - Active Target PSO
 - Adaptive PSO
 - Adaptive Mutation PSO
 - Adaptive PSO Guided by Acceleration Information
 - Attractive Repulsive Particle Swarm Optimization
 - Binary PSO
 - Cooperative Multiple PSO
 - Dynamic and Adjustable PSO
 - Extended Particle Swarms
 - ...

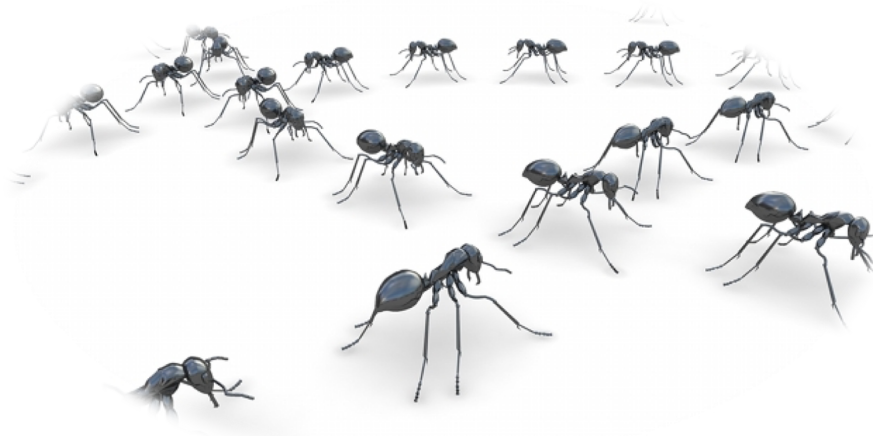
Ant Colony Optimization

Genetic Algorithms Course (ALGEN)
MSc in Electrical and Computer Engineering

Filipe Azevedo
(fta@isep.ipp.pt)

The ants

- Can explore vast areas without global view of the ground.
- Can find the food and bring it back to the nest.
- Will converge to the shortest path.

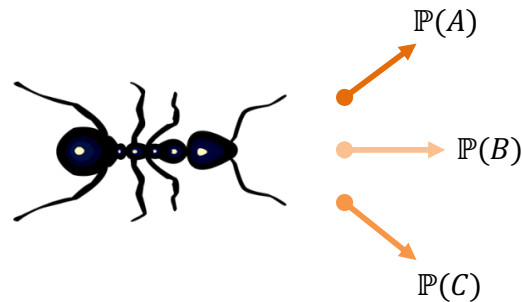


How can they manage such great tasks ?

- By leaving pheromones behind them.
- Wherever they go, they let pheromones behind here, marking the area as explored and communicating to the other ants that the way is known.

Pheromone trails

- It is well known that the primary means for ants to form and maintain the line is a pheromone trail. Ants deposit a certain amount of pheromone while walking, and each ant probabilistically prefers to follow a direction rich in pheromone.



$$\mathbb{P}(C) < \mathbb{P}(B) < \mathbb{P}(A)$$

Pheromone trails

- **Shortest path around an obstacle**
 - This elementary behavior of real ants can be used to explain how they can find the shortest path that reconnects a broken line after the sudden appearance of an unexpected obstacle has interrupted the initial path.
- Let us consider the following scenario:

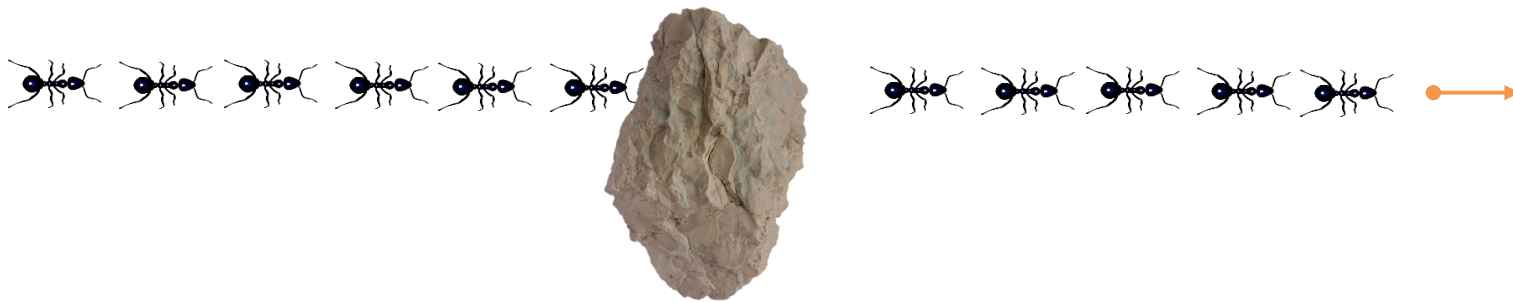
Ants are moving on a straight line that connects a food source to their nest.



Pheromone trails

- **Shortest path around an obstacle**
 - This elementary behavior of real ants can be used to explain how they can find the shortest path that reconnects a broken line after the sudden appearance of an unexpected obstacle has interrupted the initial path.

An obstacle appears on the path.



Pheromone trails

- **Shortest path around an obstacle**
 - This elementary behavior of real ants can be used to explain how they can find the shortest path that reconnects a broken line after the sudden appearance of an unexpected obstacle has interrupted the initial path.

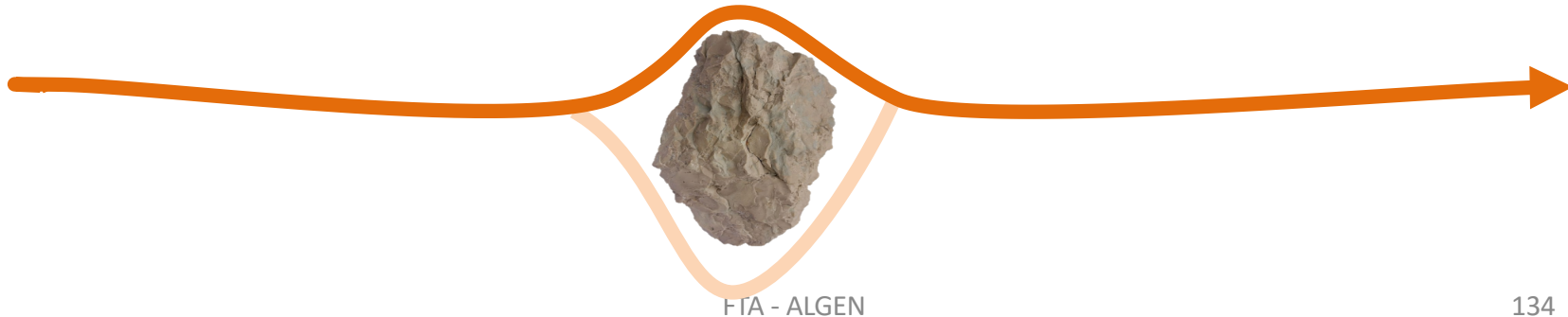
Those ants which are just in front of the obstacle cannot continue to follow the pheromone trail and therefore they have to choose between turning right or left. In this situation we can expect half the ants to choose to turn right and the other half to turn left.



Pheromone trails

- **Shortest path around an obstacle**
 - This elementary behavior of real ants can be used to explain how they can find the shortest path that reconnects a broken line after the sudden appearance of an unexpected obstacle has interrupted the initial path.

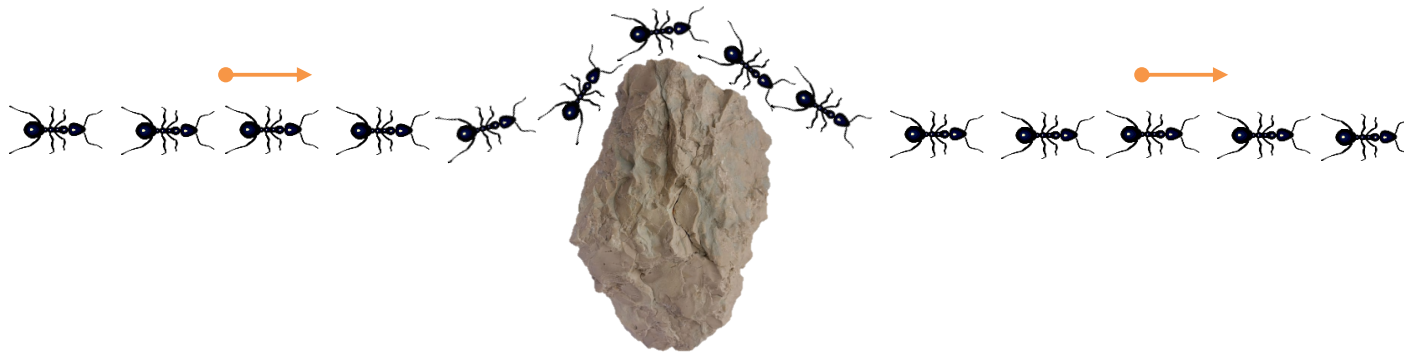
Those ants which choose, by chance, the shorter path around the obstacle will more rapidly reconstitute the interrupted pheromone trail compared to those which choose the longer path. Thus, the shorter path will receive a greater amount of pheromone per time unit and in turn a larger number of ants will choose the shorter path.



Pheromone trails

- **Shortest path around an obstacle**
 - This elementary behavior of real ants can be used to explain how they can find the shortest path that reconnects a broken line after the sudden appearance of an unexpected obstacle has interrupted the initial path.

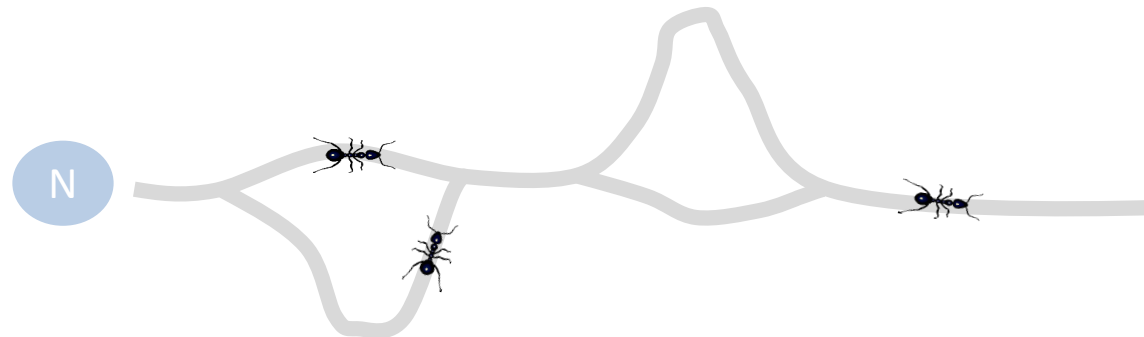
Shortest path is being obtained.



Pheromone trails

- **Shortest path from the nest to the food source**
 - Ants are able, without using any spatial Information, to identify a sudden appearance of a food source around their nest, and to find the shortest available path to it.
- Algorithm description:

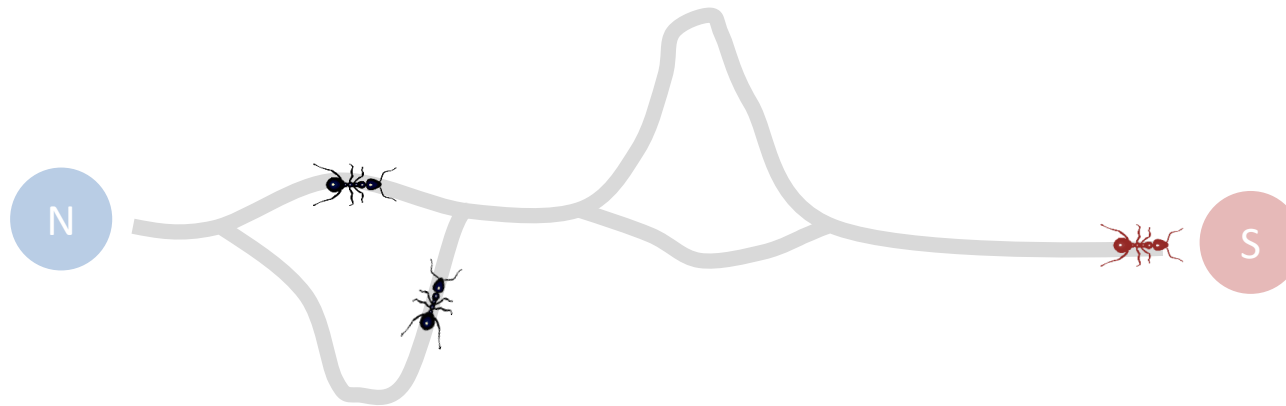
A small amount of ants travel **randomly** around the nest.



Pheromone trails

- **Shortest path from the nest to the food source**
 - Ants are able, without using any spatial information, to identify a sudden appearance of a food source around their nest, and to find the shortest available path to it.

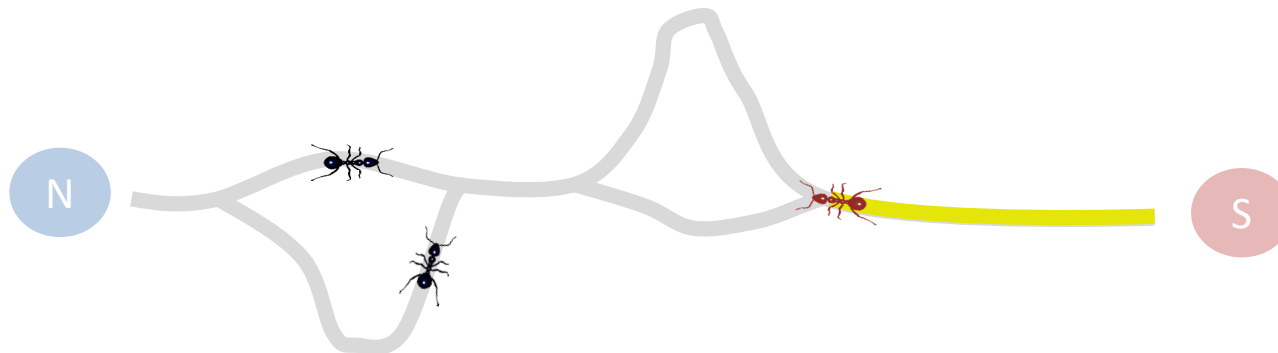
One of the ants find food source.



Pheromone trails

- **Shortest path from the nest to the food source**
 - Ants are able, without using any spatial information, to identify a sudden appearance of a food source around their nest, and to find the shortest available path to it.

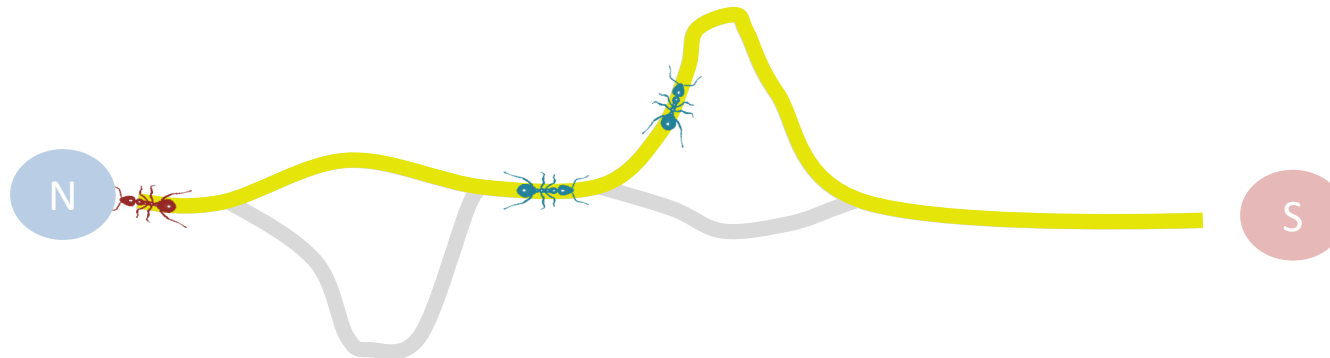
When ant finds food, it returns to the nest while **laying down pheromones trail**.



Pheromone trails

- **Shortest path from the nest to the food source**
 - Ants are able, without using any spatial Information, to identify a sudden appearance of a food source around their nest, and to find the shortest available path to it.

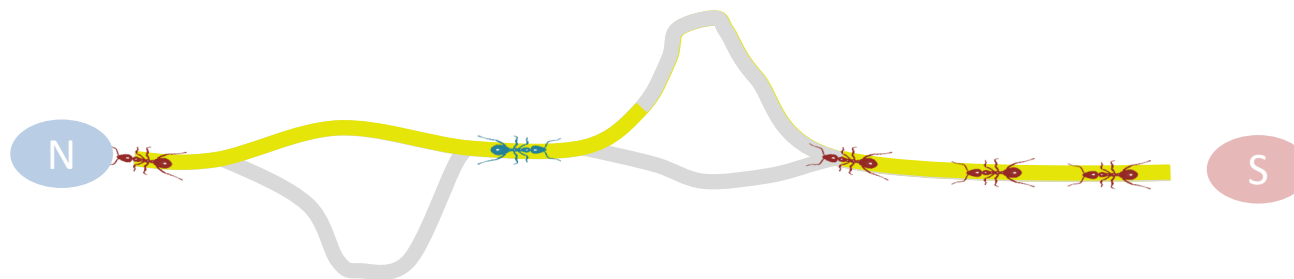
When other ants find a pheromone trail, they are likely not to keep travelling at random, but to instead follow the trail.



Pheromone trails

- **Shortest path from the nest to the food source**
 - Ants are able, without using any spatial information, to identify a sudden appearance of a food source around their nest, and to find the shortest available path to it.

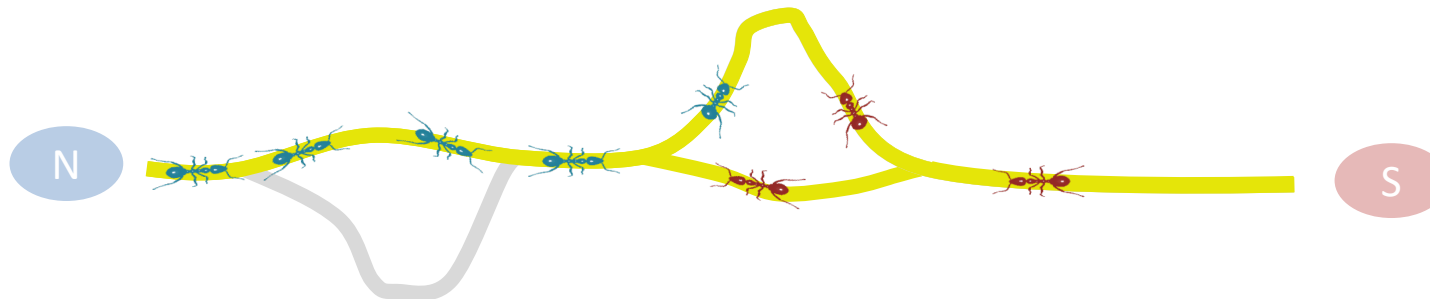
If an ant eventually find food by following a pheromone trail, it returning to the nest while **reinforcing the trail** with more pheromones.



Pheromone trails

- **Shortest path from the nest to the food source**
 - Ants are able, without using any spatial information, to identify a sudden appearance of a food source around their nest, and to find the shortest available path to it.

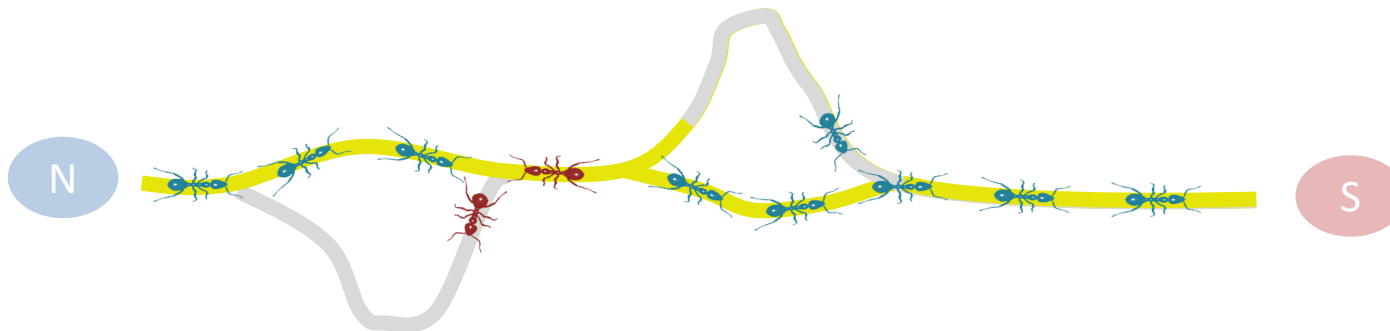
Due to their **stochastic behavior**, some ants are not following the pheromone trails, and thus uncover more possible paths.



Pheromone trails

- **Shortest path from the nest to the food source**
 - Ants are able, without using any spatial information, to identify a sudden appearance of a food source around their nest, and to find the shortest available path to it.

Over time, however, the pheromones trails starts to **evaporate**, thus reducing its attractive strength.



Pheromone trails

- **Shortest path from the nest to the food source**
 - An Intuitive proof of correctness.
 - The more time it takes for an ant to travel down the path and back again, the more time the pheromones have to evaporate.

Thus,

- **A short path, by comparison, get marched over more frequently, and thus its pheromone density becomes higher on shorter paths than longer ones.**

Pheromone evaporation also has the advantage of avoiding the convergence to a locally optimal solution. If there were no evaporation at all, all paths chosen by the first ants would tend to be excessively attractive to the following ones. In that case, the exploration of the solution space would be constrained.

Thus, when one ant finds a short path from the colony to a food source, other ants are more likely to follow that path, **and positive feedback eventually leads all the ants following a single path.**

Ant Colony Optimization algorithms

- Ant Colony Optimization (ACO) studies artificial systems that take inspiration from the behavior of real ant colonies and which are used to solve discrete optimization problems.
- Some applications:



Set partition problem

Deciding whether a given multiset of positive integers can be partitioned into two subsets A and B such that the sum of the numbers in A equals the sum of the numbers in B.

Ant Colony Optimization algorithms

- Ant Colony Optimization (ACO) studies artificial systems that take inspiration from the behavior of real ant colonies and which are used to solve discrete optimization problems.
- Some applications:



Job – Shop problems

Given a number of jobs have to be done and every job consists of using a number of machines for a certain amount of time. Find the best planning to do all the jobs on all the different machines in the shortest period of time.

Ant Colony Optimization algorithms

- Ant Colony Optimization (ACO) studies artificial systems that take inspiration from the behavior of real ant colonies and which are used to solve discrete optimization problems.
- Some applications:

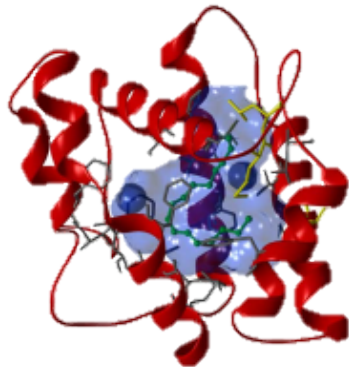


Multiple knapsack problem

Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

Ant Colony Optimization algorithms

- Ant Colony Optimization (ACO) studies artificial systems that take inspiration from the behavior of real ant colonies and which are used to solve discrete optimization problems.
- Some applications:



Protein folding

The process by which a protein structure assumes its functional shape or conformation.

Ant Colony Optimization algorithms

- Ant Colony Optimization (ACO) studies artificial systems that take inspiration from the behavior of real ant colonies and which are used to solve discrete optimization problems.
- Some applications:

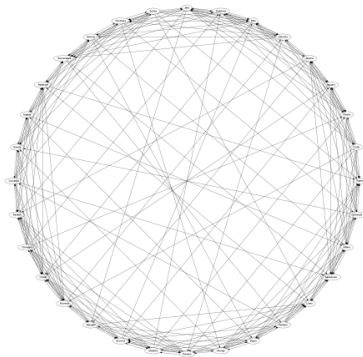


Data mining

the computational process of discovering patterns in large data sets.

Ant Colony Optimization algorithms

- Ant Colony Optimization (ACO) studies artificial systems that take inspiration from the behavior of real ant colonies and which are used to solve discrete optimization problems.
- Some applications:



Travelling salesman problem

Given a list of cities and the distances between each pair of cities, determine the shortest possible route that visits each city exactly once and returns to the origin city.

Travelling Salesman Problem

- For a complete undirected weighted graphs.
- Definitions:
 - Undirected graph $G(V, E)$ is **complete** if for every pair of vertexes $v_1, v_2 \in V$ there exist an edge $(v_1, v_2) \in E$.
 - Path P in G is a **Hamiltonian path** if P visits each vertex exactly once.
- Problem:
 - Given a complete undirected weighted graph $G(V, E)$, find the minimal Hamiltonian cycle in G .
- Complexity class:
 - **NP-Hard**

Travelling Salesman Problem

- For an undirected complete weighted graphs.
- Algorithms for **deterministic** solution:
 - **Brute force search:**
 - Compute all possible cycles and get the minimal path.
 - Time complexity: $O(|V|!)$
 - **Exact solution**
 - **Dynamic programming:**
 - Recursive algorithms such as the Held–Karp algorithm.
 - Time complexity: $O(|V|^2 2^{|V|})$
 - **Exact solution**
 - **Greedy algorithm:**
 - Such as the nearest neighbor (NN) algorithm.
 - Time complexity: $O(\log |V|)$
 - **Approximate solution**

Travelling Salesman Problem

- ACS solution
 - The Ant Colony System (ACS) is one of the ACO algorithms, which gives a **non – deterministic approximate solution** to the TSP problem with a time complexity of $O(|V|^{3 \cdot c})$, where c is a constant.

Travelling Salesman Problem

- ACS solution
 - ACS Algorithm description:
 - Given a complete undirected weighted graph $G(V, E)$:

Place $k \leq n$ ants in a randomly chosen vertexes.

Travelling Salesman Problem

- ACS solution
 - ACS Algorithm description:
 - Given a complete undirected weighted graph $G(V, E)$:

Place $k \leq n$ ants in a randomly chosen vertexes.



Each ant preforms a Hamiltonian cycle according to the **navigation function** $F(s)$.

- Ant i endowed with a working memory M_k^i , which is used to memorize vertexes already visited. At initialization, $M_k^i = \emptyset$ for every $1 \leq i \leq k$, and when ant i arrives to vertex v , vertex v is being added to M_k^i .

Travelling Salesman Problem

- ACS solution
 - ACS Algorithm description:
 - Given a complete undirected weighted graph $G(V, E)$:

Place $k \leq n$ ants in a randomly chosen vertexes.



Each ant preforms a Hamiltonian cycle according to the **navigation function** $F(s)$.

- An ant i in city r chooses the city s to move to among those which do not belong to its working memory M_k^i by applying the following probabilistic formula:

$$f(x) = \begin{cases} \arg \max_{u \notin M_k} \{\tau(r, u) \cdot \eta^\beta(r, u)\} & q < q_0 \\ s & \text{otherwise} \end{cases}$$

Travelling Salesman Problem

- $\tau(r, u)$ - Amount of pheromone trail on edge (r, u)
- $\eta(r, u)$ - A heuristic function which was chosen to be the inverse of the distance between vertexes r and u .
- β - A parameter which weighs the relative importance of pheromone trail and of closeness.
- q - A value chosen randomly with uniform probability in $[0,1]$.
- q_0 - A parameter in $[0,1]$.

Travelling Salesman Problem

- ACS solution
 - ACS Algorithm description:
 - Given a complete undirected weighted graph $G(V, E)$:

Place $k \leq n$ ants in a randomly chosen vertexes.

Each ant preforms a Hamiltonian cycle according to the **navigation function** $F(s)$.

- An ant i in city r chooses the city s to move to among those which do not belong to its working memory M_k^i by applying the following probabilistic formula:

$$f(x) = \begin{cases} \arg \max_{u \notin M_k} \{ \tau(r, u) \cdot \eta^\beta(r, u) \} & q < q_0 \\ s & \text{otherwise} \end{cases}$$

Travelling Salesman Problem

- S - A random variable selected according to the following probability distribution, which favors edges which are shorter and have a higher level of pheromone trail:

$$\mathbb{P}_k(r, s) = \begin{cases} \frac{\tau(r, s) \cdot \eta^\beta(r, s)}{\sum_{u \notin M_k} \tau(r, u) \cdot \eta^\beta(r, u)} & s \notin M_k \\ 0 & \text{otherwise} \end{cases}$$

Travelling Salesman Problem

- ACS solution
 - ACS Algorithm description:
 - Given a complete undirected weighted graph $G(V, E)$:

Place $k \leq n$ ants in a randomly chosen vertexes.



Each ant preforms a Hamiltonian cycle according to the **navigation function** $F(s)$.



While traveling, each ant applies the **local update rule** for each visited edge.

- Every time an edge is chosen by an ant, its amount of pheromone is changed according to the following formula:

$$\tau(r, s) \leftarrow (1 - \alpha) \cdot \tau(r, s) + \alpha \cdot \tau_0$$

Travelling Salesman Problem

τ_0, α - Parameters

- Local updating is intended to avoid a very strong edge being chosen by all the ants.
- Local trail updating is also motivated by trail evaporation in real ants.

Travelling Salesman Problem

- ACS solution
 - ACS Algorithm description:
 - Given a complete undirected weighted graph $G(V, E)$:

Place $k \leq n$ ants in a randomly chosen vertexes.

Each ant preforms a Hamiltonian cycle according to the **navigation function** $F(s)$.

While traveling, each ant applies the **local update rule** for each visited edge.

Once ants have completed their tours, apply the **global update rule**.

- Update the weight of each edge that belongs to the shortest tour, according to the following formula:

$$\tau(r, s) \leftarrow (1 - \alpha) \cdot \tau(r, s) + \alpha \cdot L_{min}^{-1}$$

Travelling Salesman Problem

L_{min} - The length of the shortest tour.

- Global updating is intended to reward edges belonging to shorter tours.
- The amount of weight deposited on each edge is inversely proportional to the length of the tour: The shorter the tour the greater the amount of weight deposited on edges.

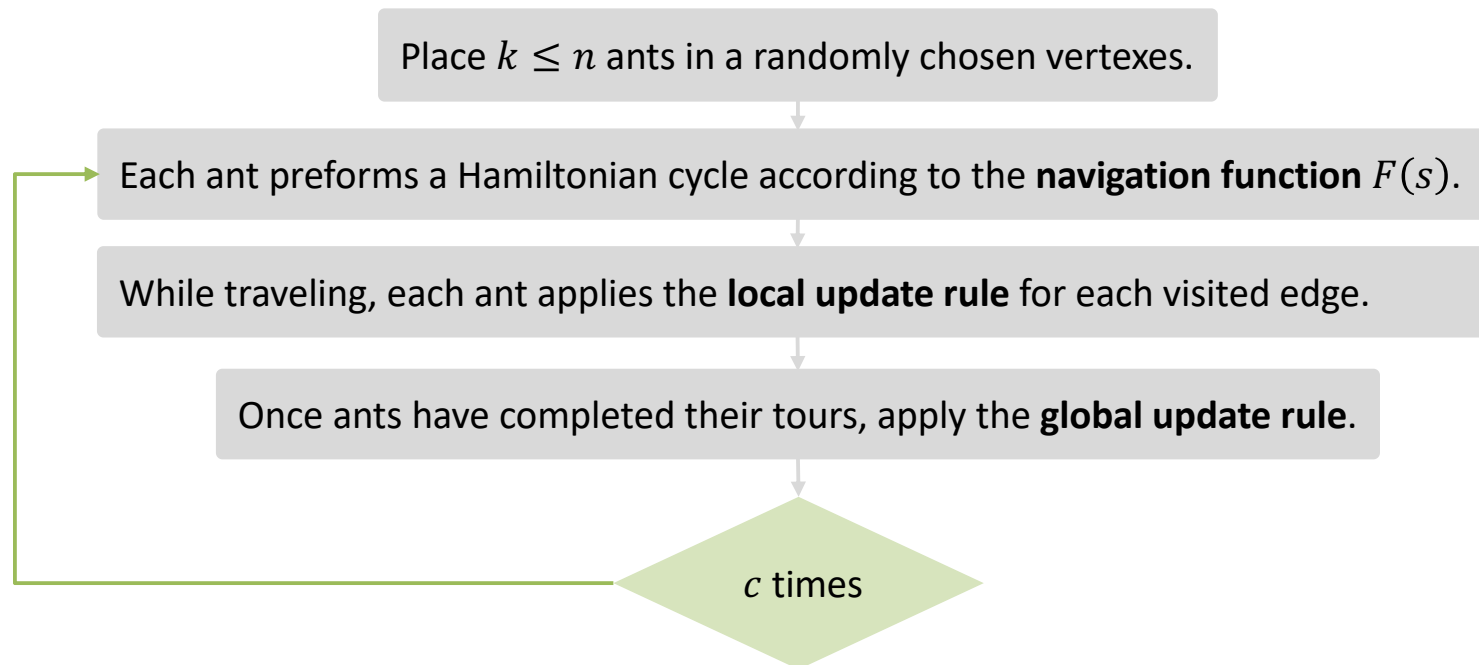
Travelling Salesman Problem

L_{min} - The length of the shortest tour.

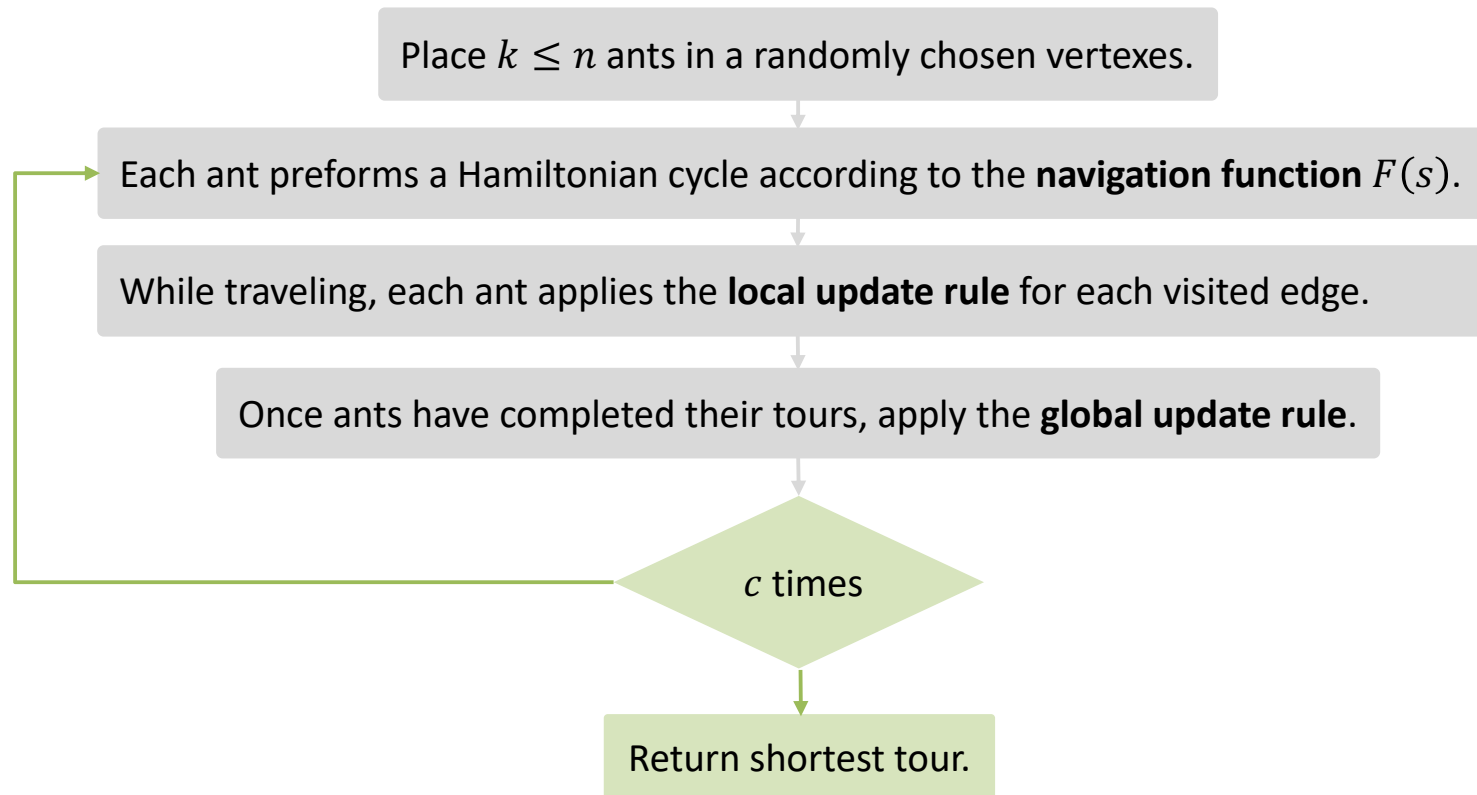
- Global updating is intended to emulate the property of differential pheromone trail accumulation, which in the case of real ants was due to the interplay between length of the path and continuity of time.

Travelling Salesman Problem

- ACS solution
 - ACS Algorithm description:
 - Given a complete undirected weighted graph $G(V, E)$:



Travelling Salesman Problem

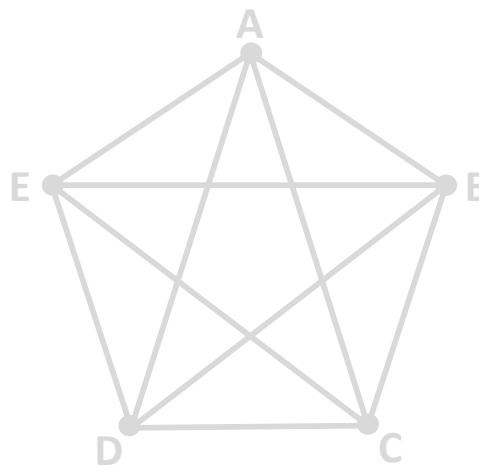


Travelling Salesman Problem

- For an undirected complete weighted graphs:
- Algorithm – Initialization

Choose constant k between 1 and $|V|$ to be the number of ants.

$$k = 3$$

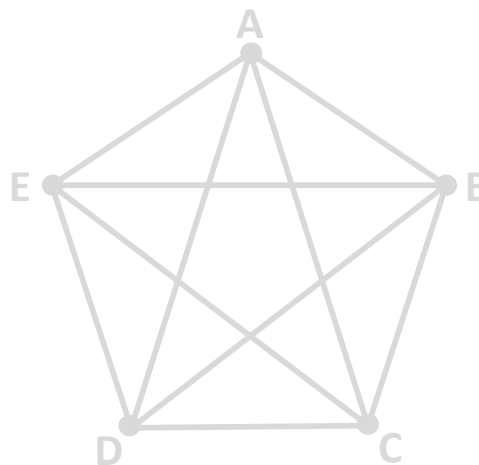


Travelling Salesman Problem

- For an undirected complete weighted graphs:
- Algorithm – Initialization

Place each ant in a randomly chosen city.

$$k = 3$$

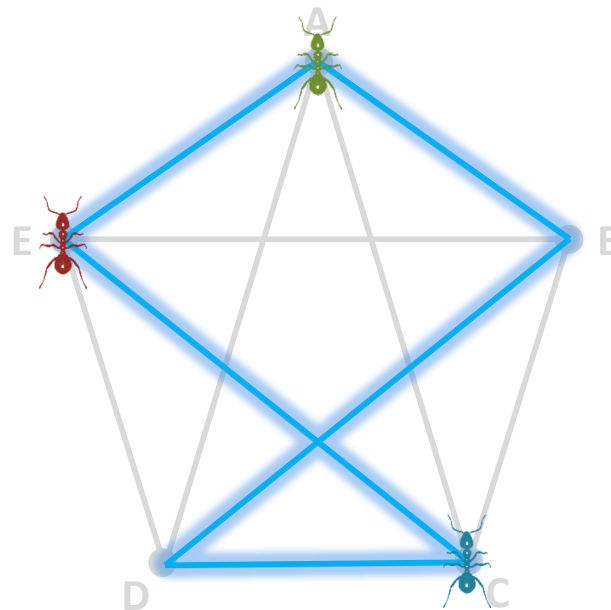


FTA - ALGEN

Travelling Salesman Problem

- For an undirected complete weighted graphs:
- Algorithm – Step 2

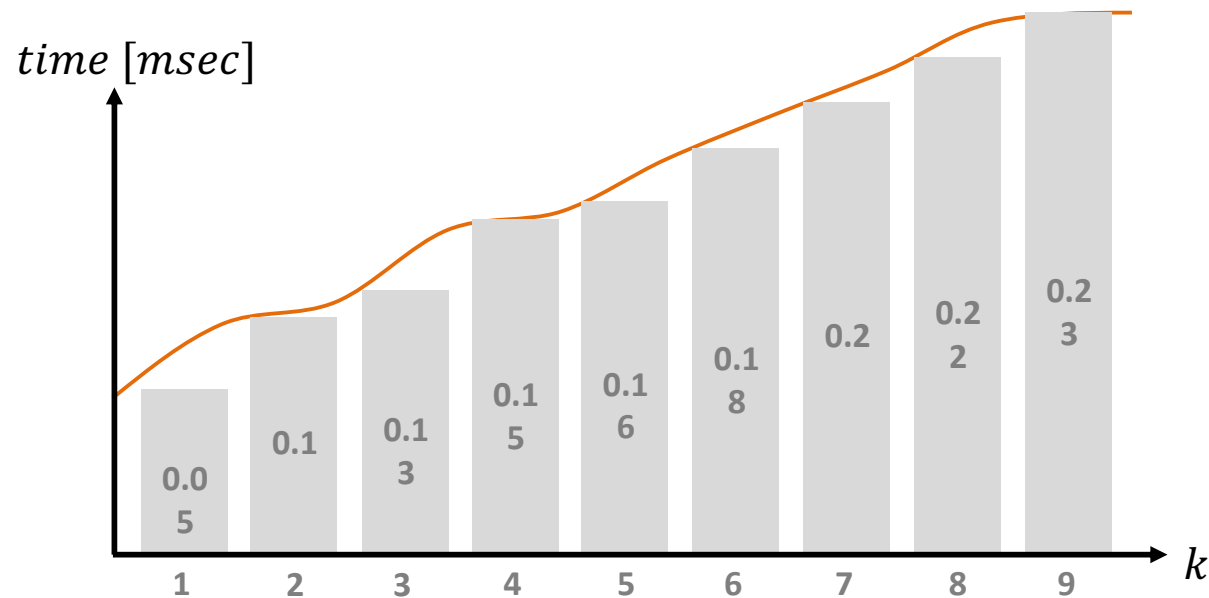
Apply the **Global Update Rule**.



FTA - ALGEN

Travelling Salesman Problem

- For an undirected complete weighted graphs:
- Number of ants
 - ACS runtime on a TSP problem with 10 vertexes for $1 \leq k < 10$, where k is the number of ants.



Travelling Salesman Problem

- For an undirected complete weighted graphs:
- An Intuitive proof of correctness
 - We can interpret the ant colony as a reinforcement learning system, in which reinforcements modify the strength (Pheromone trail) of connections between vertexes.
 - Each ant has two options:

Exploit the experience accumulated by the ant colony in the form of pheromone trail (pheromone trail will tend to grow on those edges which belong to short tours, making them more desirable).

Apply a biased **exploration of new paths** (exploration is biased towards short and high trail edges) by choosing the vertex to move to randomly, with a probability distribution that is a function of both the accumulated pheromone trail, the heuristic function, and the working memory M_k .



Bibliography

Genetic Algorithms Course (ALGEN)
MSc in Electrical and Computer Engineering

Filipe Azevedo
(fta@isep.ipp.pt)

Important Teaching Material

1. David E. Goldberg, Addison Wesley , *“Genetic Algorithms in Search, Optimization & Machine Learning”*, 1989
2. Melanie Mitchell, *“An Introduction to Genetic Algorithms”*, MIT Press, 1997
3. Lawrence Davis, Van Nostrand Reinhold, *“Handbook of Genetic Algorithms”*, New York, 1991
4. Z. Michalewicz, *“Genetic Algorithms + Data Structures = Evolution Programs”*, Springer-Verlag, 1994
5. Kennedy, Eberhart, *“Swarm Intelligence”*, Morgan Kaufmann Publishers, 2001
6. Kenneth A. De Jong, *“Evolutionary Computation”*, MIT Press, A Bradford Book, 2006
7. A.E. Eiben, James E Smith, *“Introduction to Evolutionary Computing (Natural Computing Series)”*, Springer; 2nd ed. 2015

Complementary Teaching Material

1. Evolutionary Computation MIT Press, 1993-Present, Impact factor 3.600
2. IEEE Transactions on Evolutionary Computation IEEE Press, 1997-Present, Impact factor 5.908
3. Genetic Programming and Evolvable Machines Springer, 2000-Present, Impact factor 1.143
4. Evolutionary Intelligence Springer, 2008-Present
5. Journal of Artificial Evolution and Applications Hindawi, 2008-2010
6. Memetic Computing Springer, 2009-Present, Impact factor 0.900
7. International Journal of Applied Evolutionary Computation IGI Global, 2010-Present
8. Swarm and Evolutionary Computation Elsevier, 2011-Present, Impact factor 2.963
9. International Journal of Swarm Intelligence and Evolutionary Computation OMICS group, 2012-Present
10. <http://www.b-on.pt>

Genetic Algorithms Course (ALGEN)

MSc in Electrical and Computer Engineering

Filipe Azevedo
(fta@isep.ipp.pt)